VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FACULTY OF ELECTRICAL-ELECTRONICS ENGINEERING
DEPARTMENT OF ELECTRONICS

LABORATORY REPORT

COMPUTER ARCHITECTURE (EE3043)

Class: L01 - Group: 24 - Semester: 251

Milestone 2

Design of a Single Cycle RISC-V Processor

Instructor: MS. Trần Hoàng Linh

Date: November 08, 2025

TASK ASSIGNMENT TABLE

Full name	Student ID	Task	Grading
Võ Phan Tiến Đạt	2310716	Coding (Chính) Simulation (Chính) Report writing	100%
Lê Minh Tiến	2313427	Coding (Chính) Simulation (Chính) Report writing	100%
Đào Thiên Phước	2312766	Coding	5%

Design of a Single Cycle RISC-V Processor

1. Introduction

Trong Milestone 2 này, nhóm chúng em sẽ thiết kế một CPU đơn chu kỳ. CPU này triển khai Kiến trúc Tập lệnh (ISA) RV32I 32 bit.

Các thiết bị ngoại vi tiêu chuẩn được bộ xử lý này hỗ trợ bao gồm LCD, LED, Công tắc và LED 7 đoạn.

Về bộ nhớ, bộ xử lý sử dụng hai mô hình bộ nhớ giống hệt nhau về mặt logic. Một là IMEM (Bộ nhớ Lệnh) chỉ đọc, trong khi mô hình còn lại là DMEM (Bộ nhớ Dữ liệu) đọc-ghi.

Thiết kế được tuân thủ các quy định và thông số kỹ thuật được nêu mô tả trong file Milestone 2 và được kiểm tra bằng một testbench toàn diện do TA cung cấp.

2. Design Strategy

2.1. Overview

Nhóm sẽ tuân thủ nghiêm ngặt thiết kế do môn học cung cấp trong file Milestone 2. Tuần 1 sẽ dành riêng cho việc thiết kế ALU và BRC. Tuần 2 bao gồm thiết kế Tập Thanh ghi (Regfile), Bộ nhớ (bao gồm Imem, Dmem) và LSU. Tuần 3 sẽ bao gồm thiết kế Immediate generation, Control Unit và Top Module (toàn bộ Single Cycle CPU), được thể hiện trong Hình 1. Trong Tuần 4, nhóm sẽ tiến hành mô phỏng trên server và chạy các chương trình ứng dụng trên bộ FPGA DE2 để hoàn thành báo cáo.

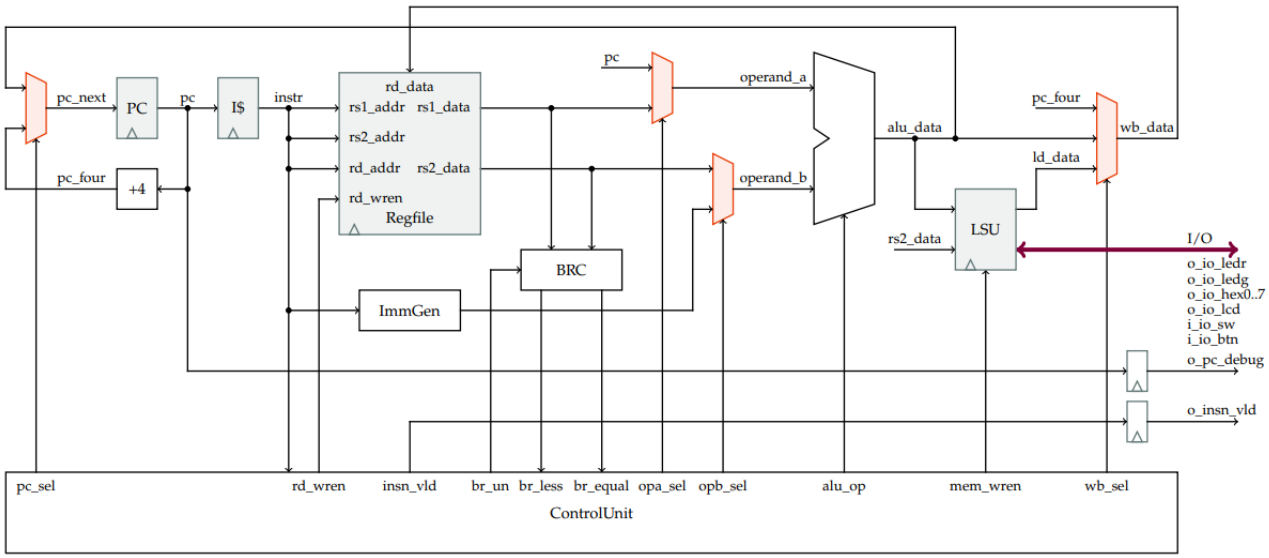


Figure 1: Single Cycle Processor

Lưu ý quan trọng trong quá trình thiết kế: Không sử dụng các toán tử SystemVerilog như cộng/trừ (+/-), so sánh (<, >), dịch chuyển (<<, >>), nhân (*), chia (/), lấy dư (%) và các toán tử không thể synthesis khác trong thiết kế.

2.2. Arithmetic Logic Unit (ALU)

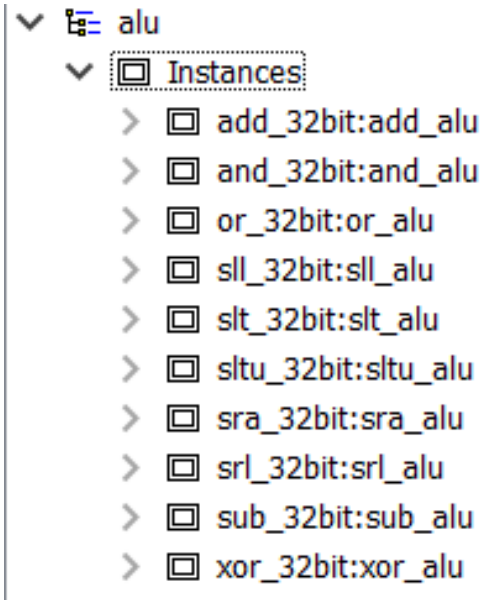
ALU là khối có khả năng thực hiện các phép toán số học và logic, khối này sẽ nhận hai đầu vào 32 bit, thực hiện phép toán và đưa ra kết quả 32 bit. Điều này được định nghĩa trong tập lệnh RV32I như hình bên dưới.

alu_op	Description (R-type)	Description (I-type)
ADD	$rd \leftarrow rs1 + rs2$	$rd \leftarrow rs1 + imm$
SUB	$rd \leftarrow rs1 - rs2$	n/a
SLT	$rd \leftarrow (rs1 < rs2)?\ 1:0$	$rd \leftarrow (rs1 < imm)?\ 1:0$
SLTU	$rd \leftarrow (rs1 < rs2)?\ 1:0$	$rd \leftarrow (rs1 < imm)?\ 1:0$
XOR	$rd \leftarrow rs1 \oplus rs2$	$rd \leftarrow rs1 \oplus imm$
OR	$rd \leftarrow rs1 \vee rs2$	$rd \leftarrow rs1 \vee imm$
AND	$rd \leftarrow rs1 \wedge rs2$	$rd \leftarrow rs1 \wedge imm$
SLL	$rd \leftarrow rs1 \ll rs2[4:0]$	$rd \leftarrow rs1 \ll imm[4:0]$
SRL	$rd \leftarrow rs1 \gg rs2[4:0]$	$rd \leftarrow rs1 \gg imm[4:0]$
SRA	$rd \leftarrow rs1 \ggg rs2[4:0]$	$rd \leftarrow rs1 \ggg imm[4:0]$

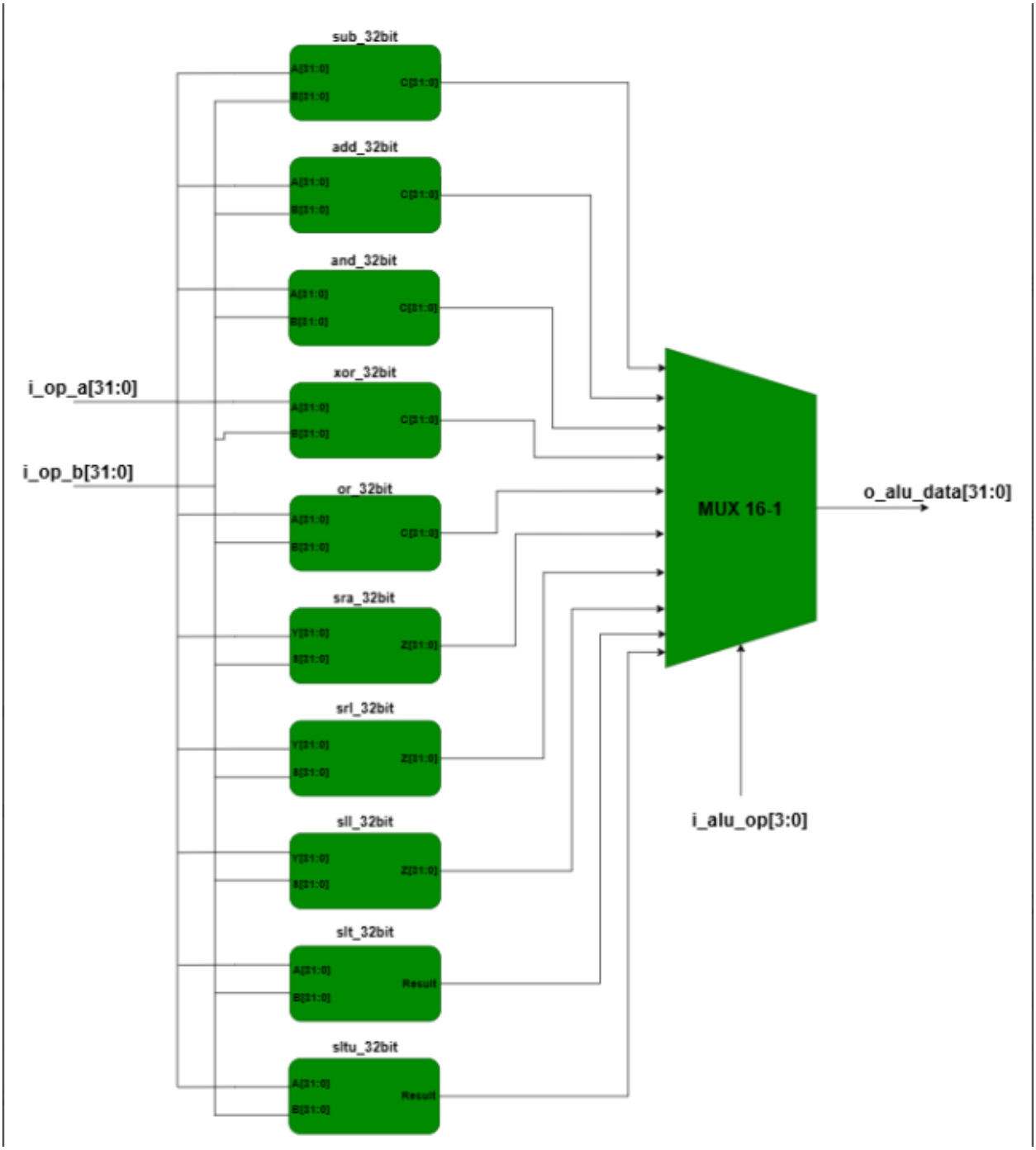
2.2.1. Design ideas

Nhóm sẽ thiết kế tuần tự từng khối chức năng trong ALU. các mô-đun như add_32bit.sv, sub_32bit.sv, xor_32bit.sv, or_32bit.sv, sra_32bit.sv, srl_32bit.sv, sll_32bit.sv, slt_32bit.sv, sltu_32bit.sv và nối chúng vào một bộ MUX lớn để tạo thành bộ alu.sv.

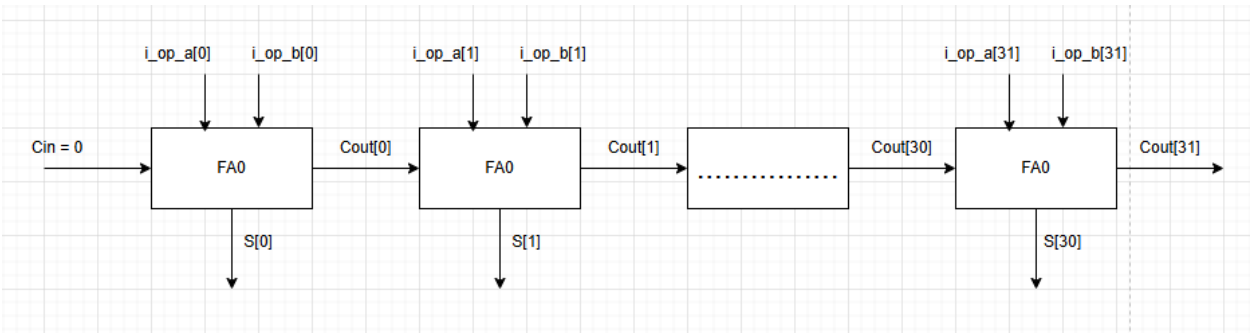
Ngoài ra, một vài module phụ khác cũng cần thiết để khởi tạo trong việc xây dựng các khối khác như full_adder.sv, mux_32_to_1.sv. Sẽ có một MUX 16-1 để chọn loại toán tử cần thực hiện. MUX này sẽ được lựa chọn bởi tín hiệu từ Bộ Điều khiển.



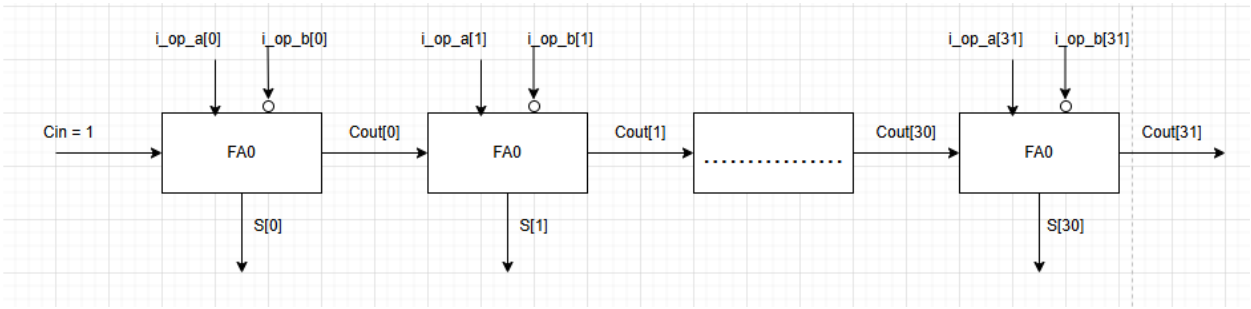
Design sketch: thiết kế của khối ALU được như hình ảnh bên dưới.



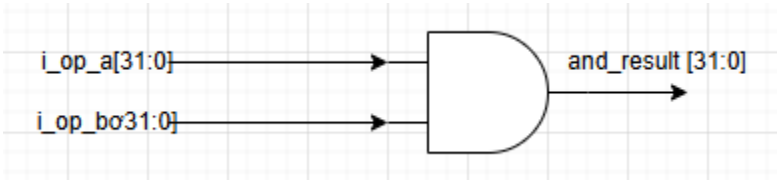
Thiết kế của bộ cộng: Theo phương pháp RCA.



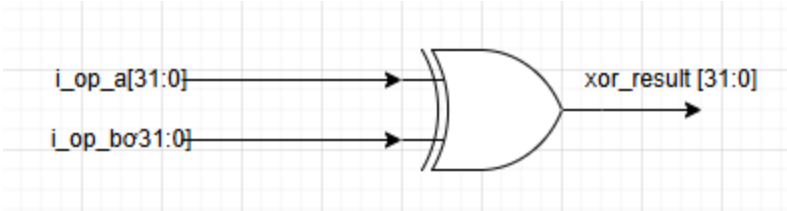
Thiết kế bộ trừ: Cũng theo phương pháp RCA.



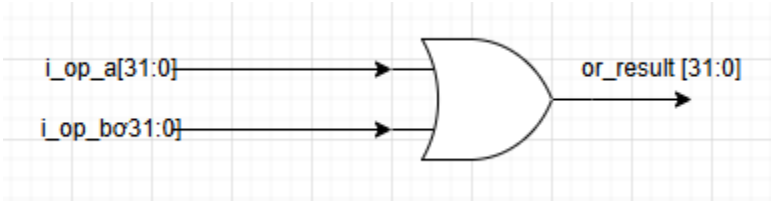
Thiết kế bộ AND:



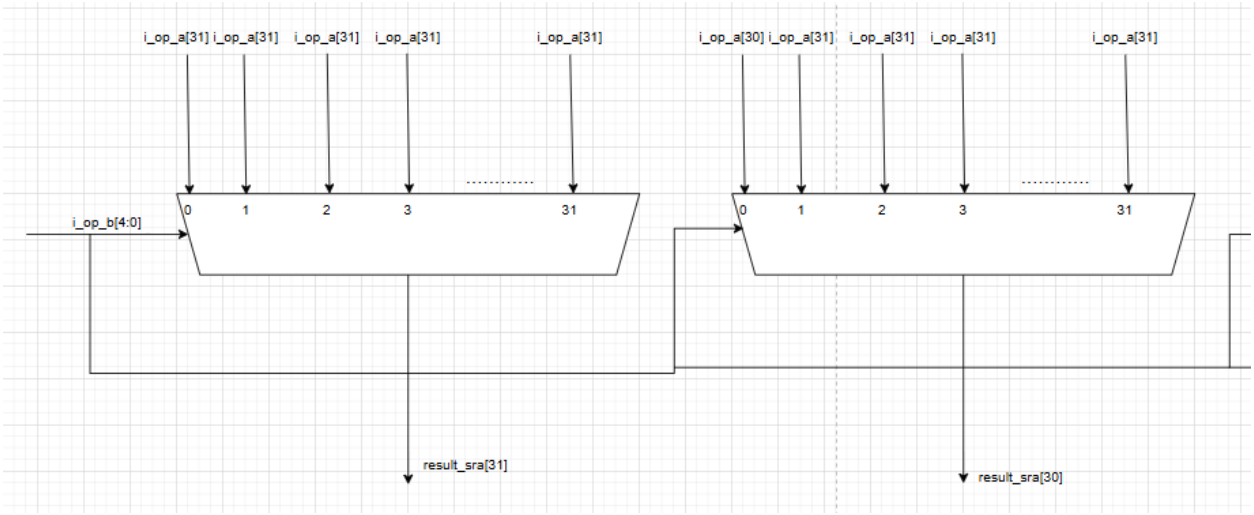
Thiết kế bộ XOR:

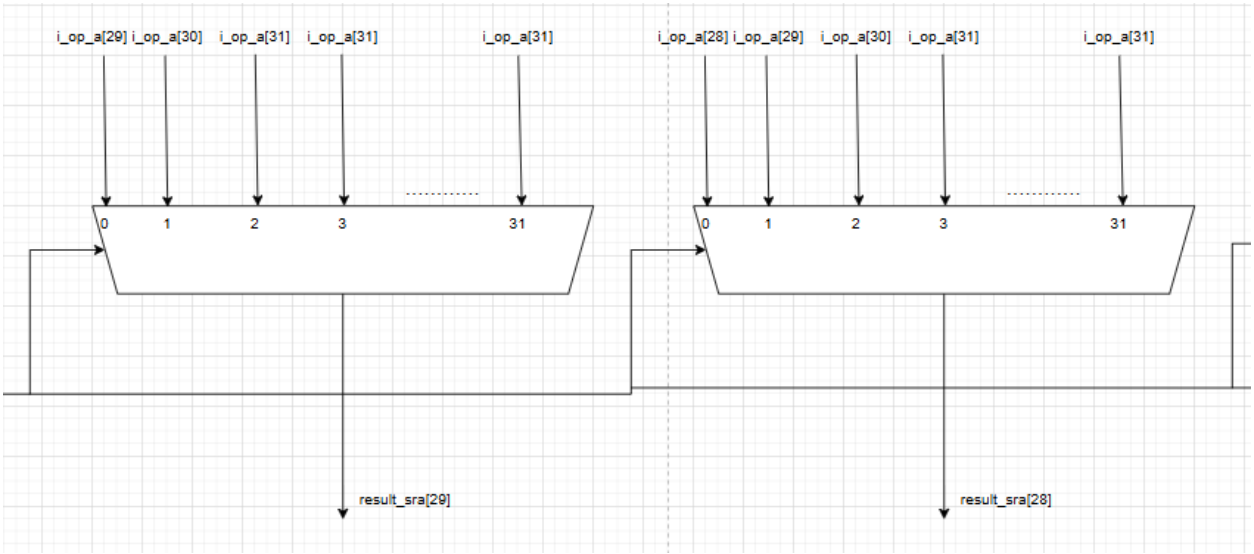


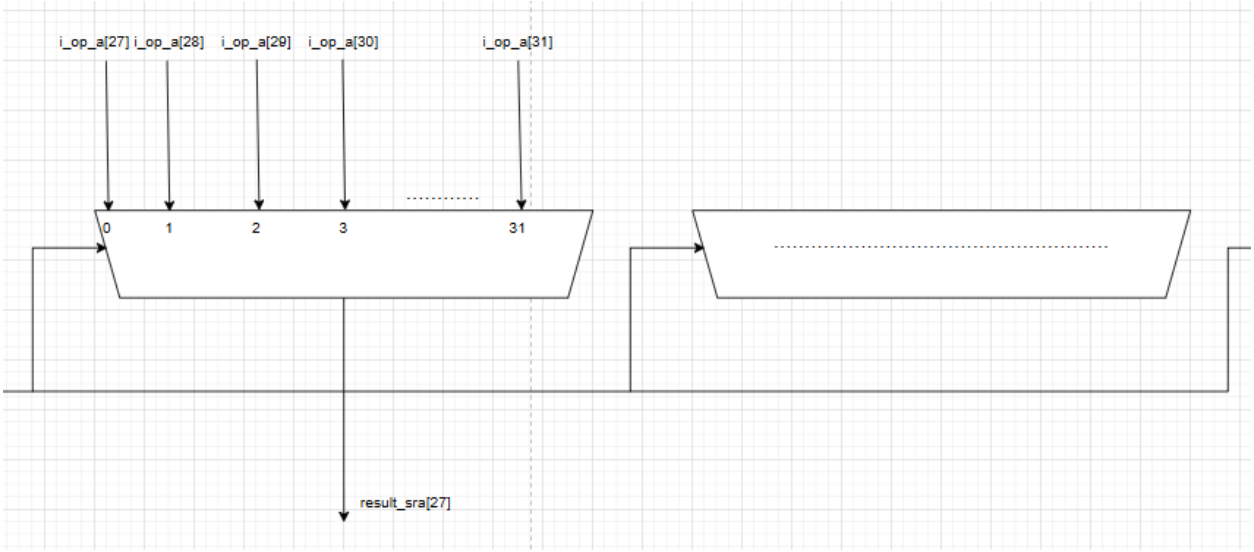
Thiết kế bộ OR:

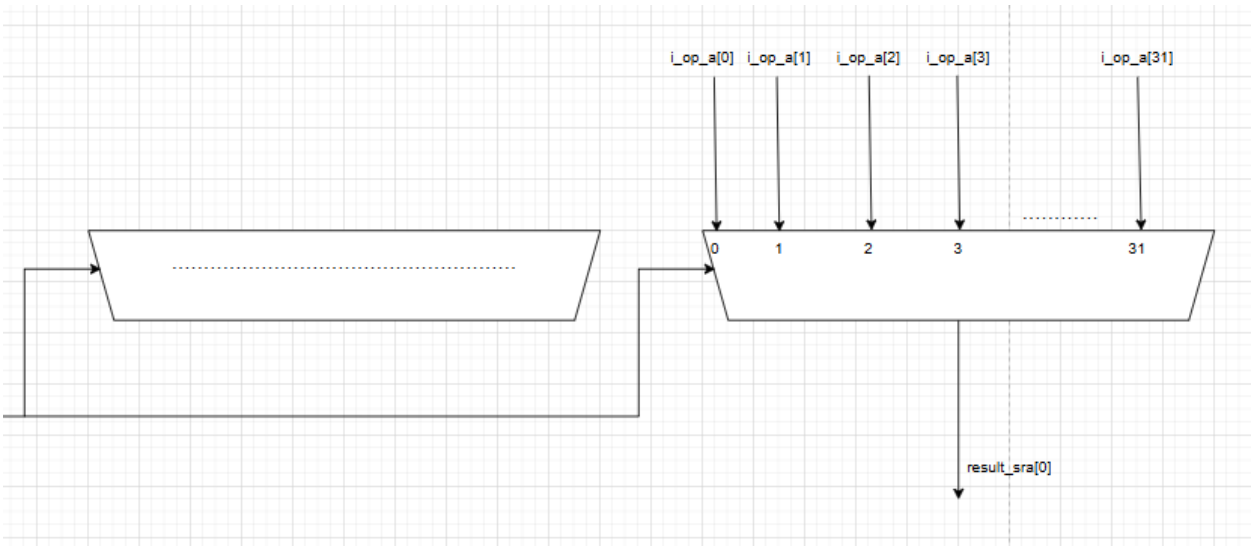


Thiết kế bộ SRA: Gồm các bộ MUX32-1 ghép song song nhau.


i_op_a[31] i_op_a[31] i_op_a[31] i_op_a[31] i_op_a[31]
i_op_a[30] i_op_a[31] i_op_a[31] i_op_a[31] i_op_a[31]
i_op_b[4:0]
0 1 2 3 31
0 1 2 3 31
result_sra[31]
result_sra[30]

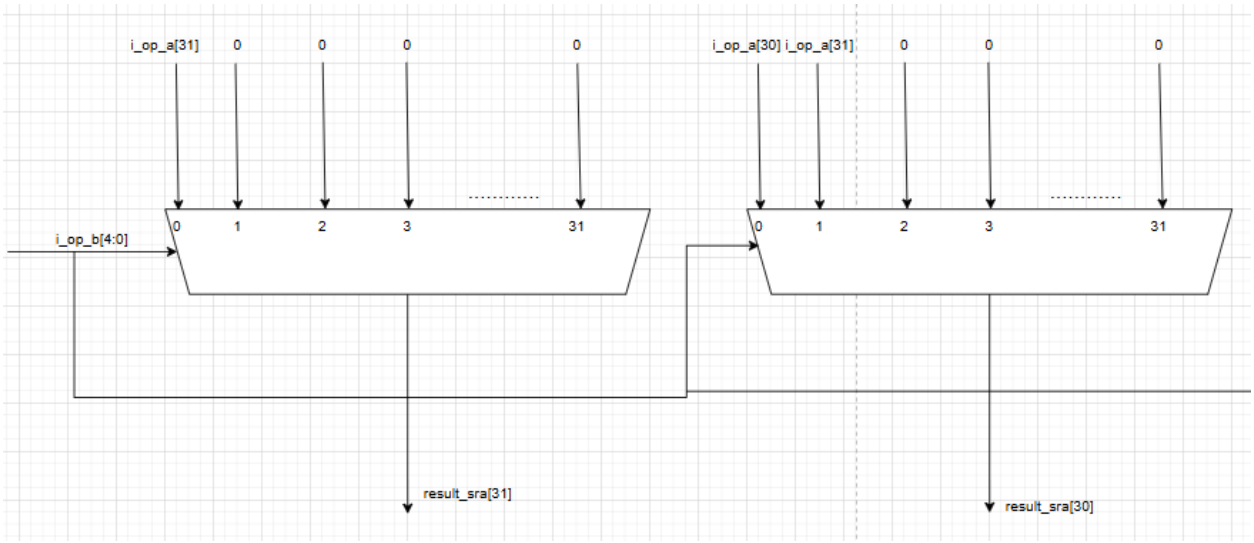

i_op_a[29] i_op_a[30] i_op_a[31] i_op_a[31] i_op_a[31]
i_op_a[28] i_op_a[29] i_op_a[30] i_op_a[31] i_op_a[31]
0 1 2 3 31
0 1 2 3 31
result_sra[29]
result_sra[28]

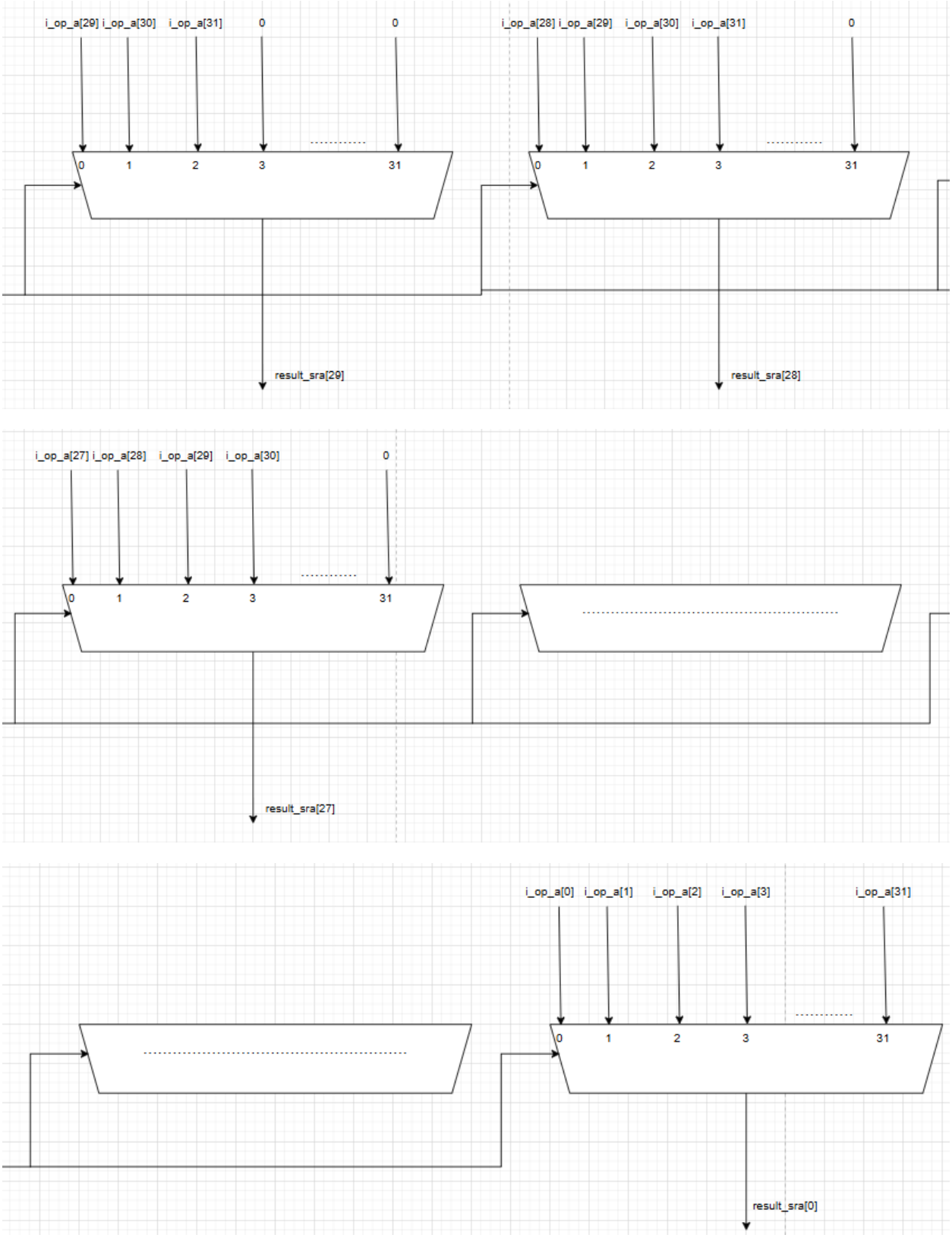

i_op_a[27] i_op_a[28] i_op_a[29] i_op_a[30] i_op_a[31]
0 1 2 3 31
result_sra[27]



*Chú thích: Dấu "…" thể hiện các ngõ vào hoặc các mux được ẩn đi để hình vẽ gọn hơn.

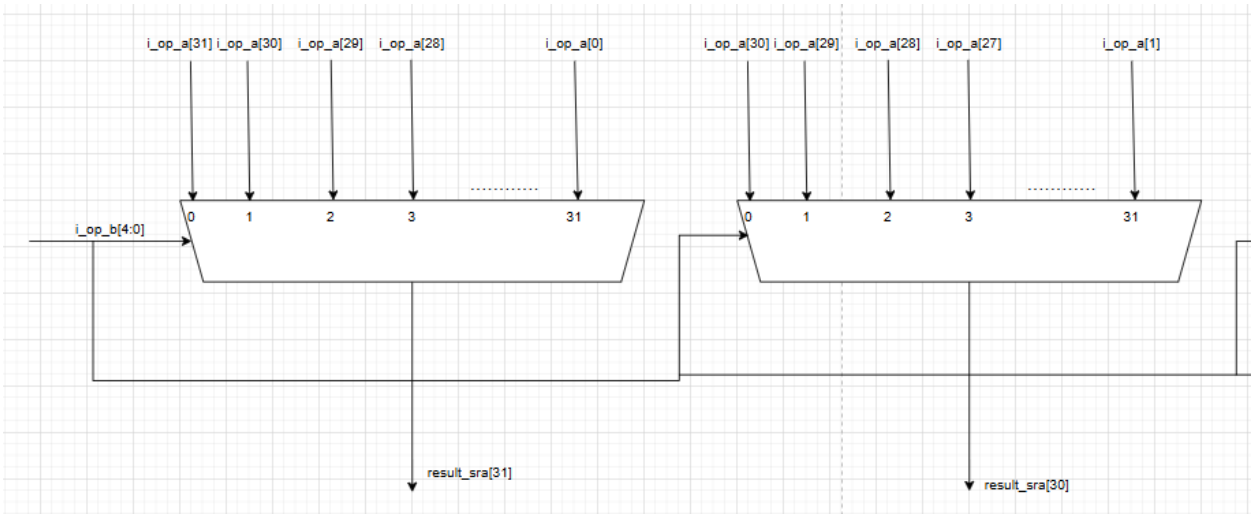
Thiết kế bộ SRL: Cấu trúc tương tự như SRA.

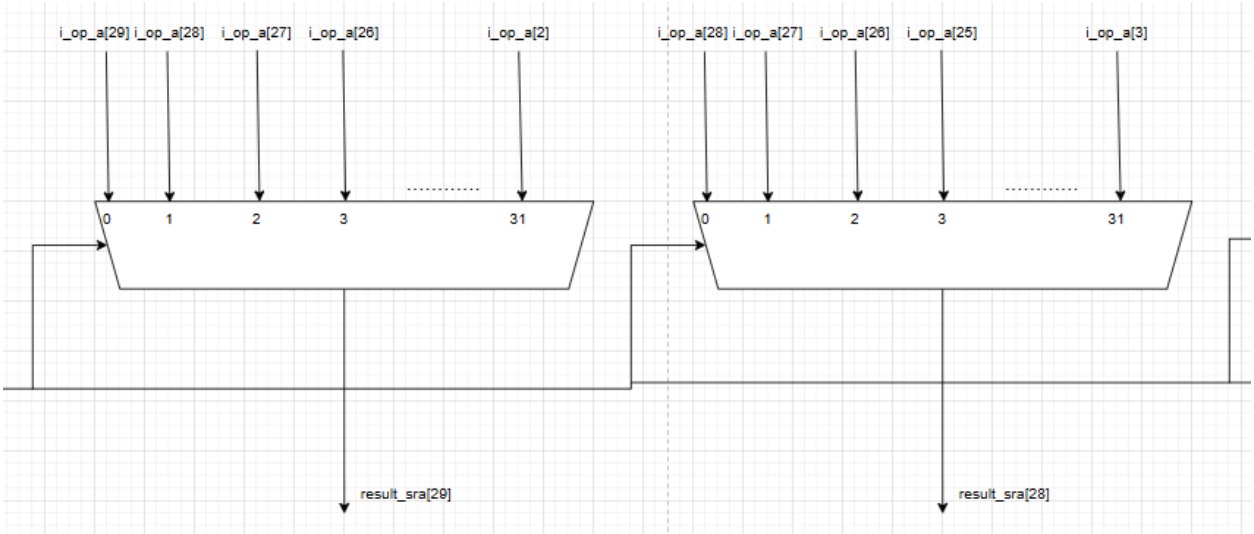


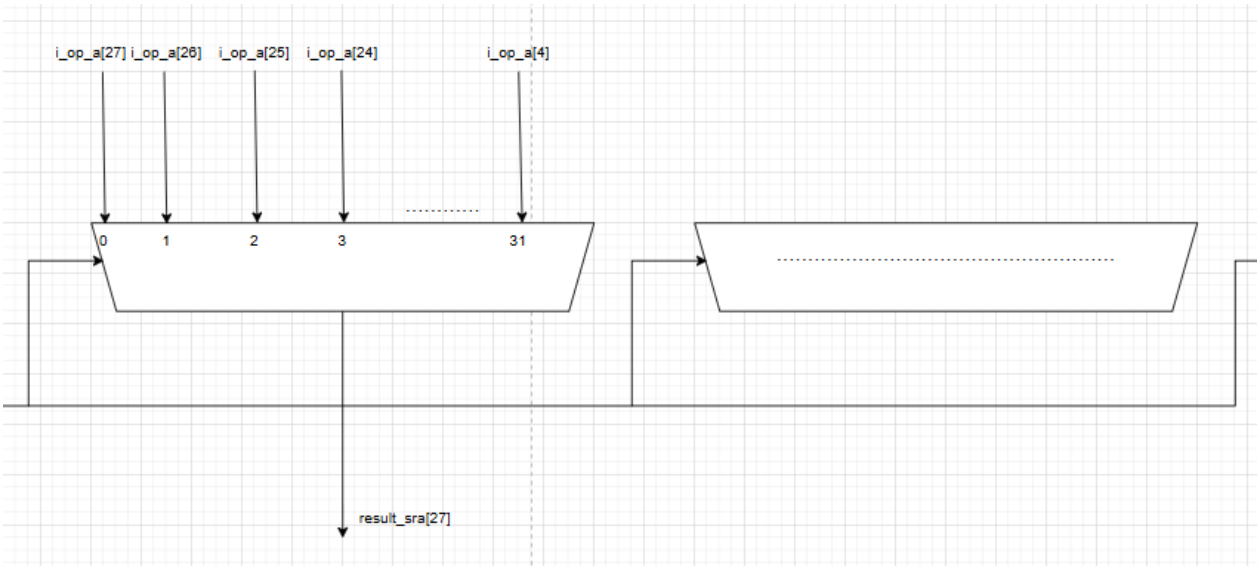


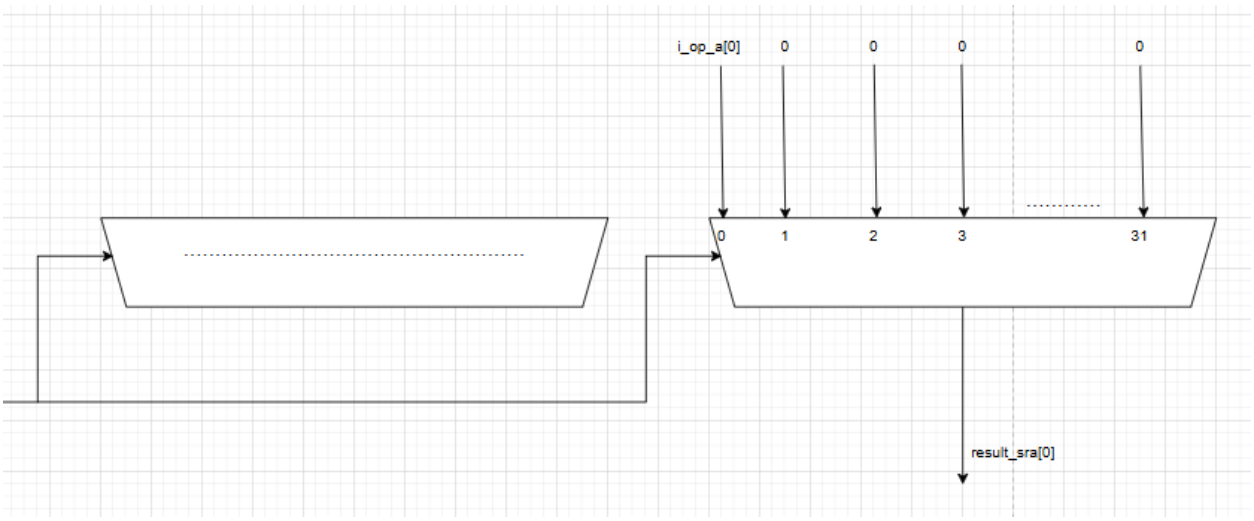
*Chú thích: Dấu "…" thể hiện các ngõ vào hoặc các mux được ẩn đi để hình vẽ gọn hơn.

Thiết kế bộ SLL: Cấu trúc tương tự như các bộ dịch phải.









Thiết kế bộ so sánh: slt và sltu được khởi tạo từ module BRC (sẽ được trình bày chi tiết ở mục sau) trong đó các ngõ vào i_op_a và i_op_b được nối lần lượt với các ngõ vào của module BRC là i_rs1_data và i_rs2_data. Ngõ vào i_br_un được điều khiển bởi Control Unit (kết hợp với tín hiệu lựa chọn operations), ví dụ, nếu lựa chọn toán tử slt thì i_br_un = 0, ngược lại, chọn sltu thì i_br_un = 1. Ngõ ra o_alu_data được nối với bộ MUX lớn để lựa chọn ngõ ra ALU.

2.2.2. Specification

Module name: alu.sv

Signal	Width	Direction	Description
i_op_a	32	Input	First operand for ALU operations.
i_op_b	32	Input	Second operand for ALU operations.
i_alu_op	4	Input	The operation to be performed.
o_alu_data	32	Output	Result of the ALU operation.

2.3. Branch Comparison Unit (BRC) (Bổ sung cái này)

Đơn vị BRC chịu trách nhiệm so sánh giá trị của hai thanh ghi để xác định kết quả của các lệnh rẽ nhánh. Đơn vị này cần có khả năng xử lý cả phép so sánh có dấu và không dấu, do đó cần thêm tín hiệu i_br_un để xác định đầu ra.

Thiết kế bộ BRC được trình bày như hình vẽ:

2.3.1. Specification

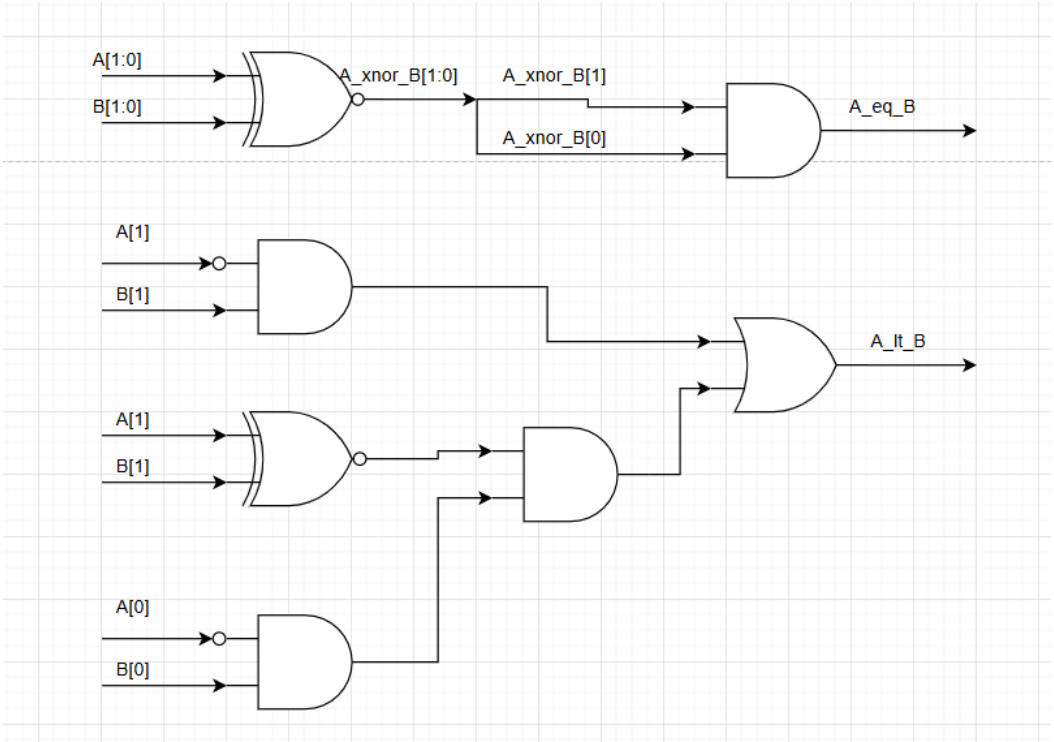
Module name: brc.sv

Signal	Width	Direction	Description
i_rs1_data	32	Input	Data from the first register
i_rs2_data	32	Input	Data from the second register
i_br_un	1	Input	Comparison mode (1 if signed, 0 if unsigned)
o_br_less	1	Output	Output is 1 if rs1 < rs2
o_br_equal	1	Output	Output is 1 if rs1 = rs2

Thiết kế bộ BRC được xây dựng từ việc thiết kế bộ so sánh hai không dấu số 2 bit, rồi sau đó thiết kế bộ so sánh không dấu 2 số 4 bit, 8 bit và 16 bit. Sử dụng hai bộ so sánh không dấu hai số 16 bit để thiết kế bộ BRC.

Được trình bày như sau:

- Thiết kế bộ so sánh không dấu hai số 2 bit:

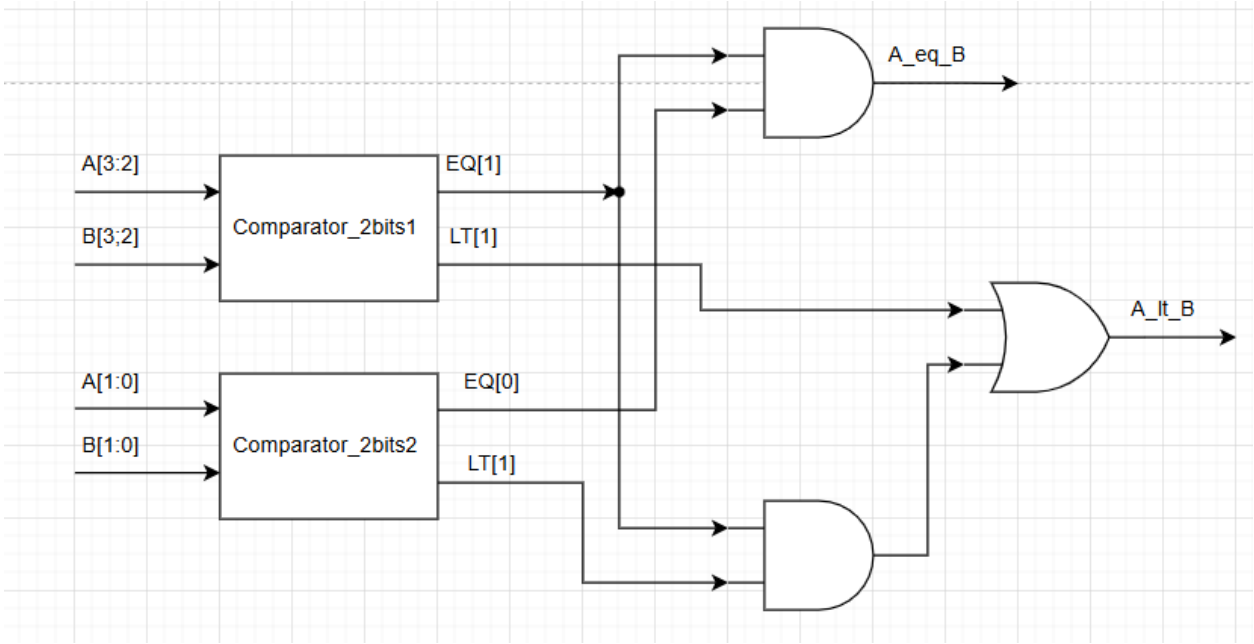


Sử dụng cổng so trong việc tạo tín hiệu so sánh bằng. XOR từng bit của 2 số để xét sự giống nhau của từng cặp bit và AND hai kết quả, nếu cả hai cặp bit đều giống nhau thì A_eq_B = 1.

A_lt_B = 1 khi A[1] = 0 và B[1] = 1 (lưu ý: so sánh không dấu). Nếu bit A[1] = B[1], thì ta xét nếu A[0] = 0 và B[0] = 1 thì A_lt_B = 1.

- Thiết kế bộ so sánh không dấu 4 bit:

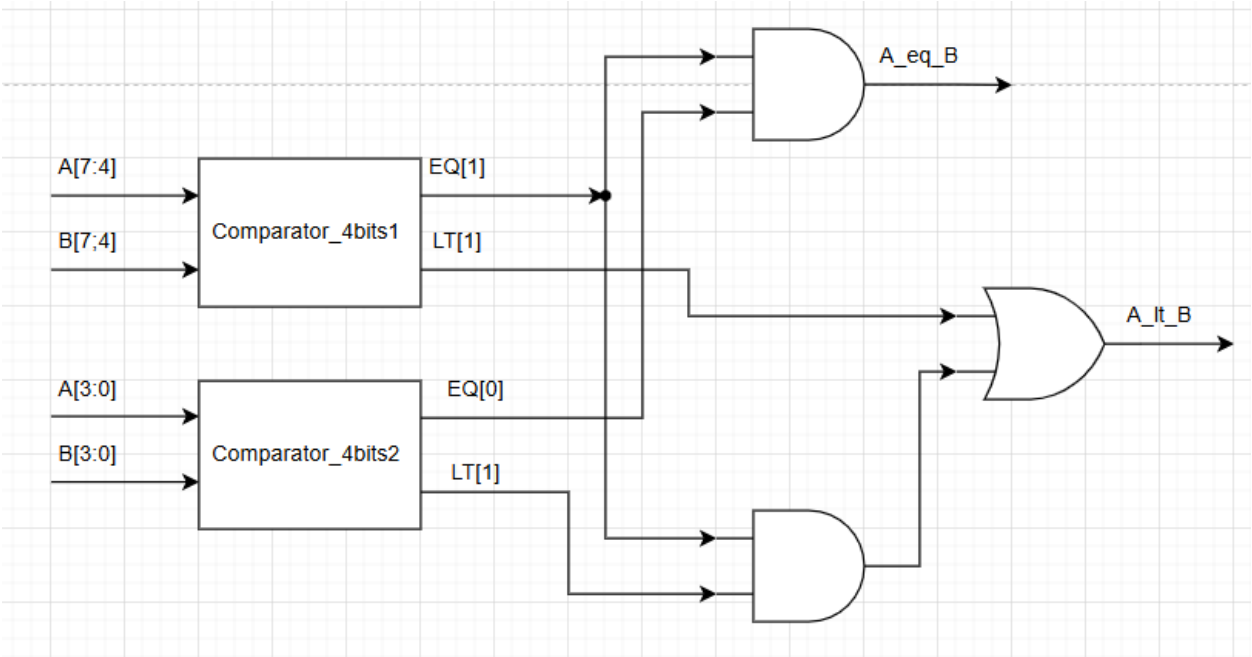
  Sử dụng 2 bộ so sánh số 2 bit vừa thiết kế:



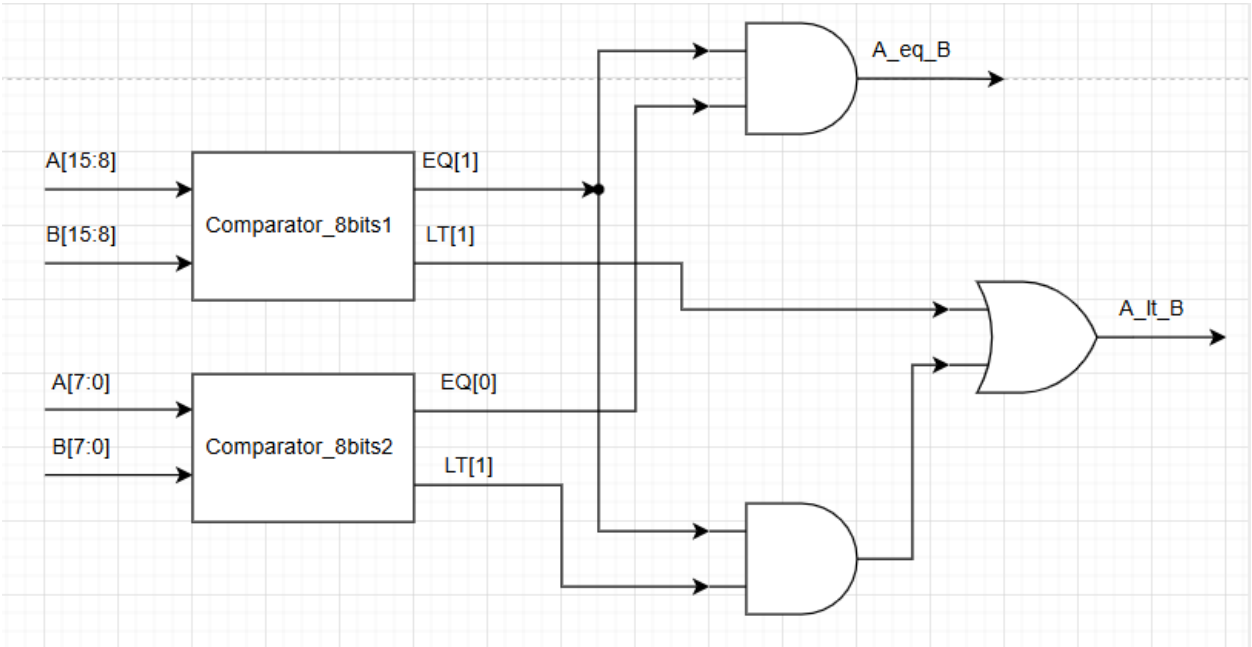
  A_eq_B = 1 khi A[3:2] = B[3:2] và A[1:0] = B[1:0]

  A_lt_B = 1 khi A[3:2] < B[3:2] hoặc A[3:2] = B[3:2] và A[1:0] < B[1:0]

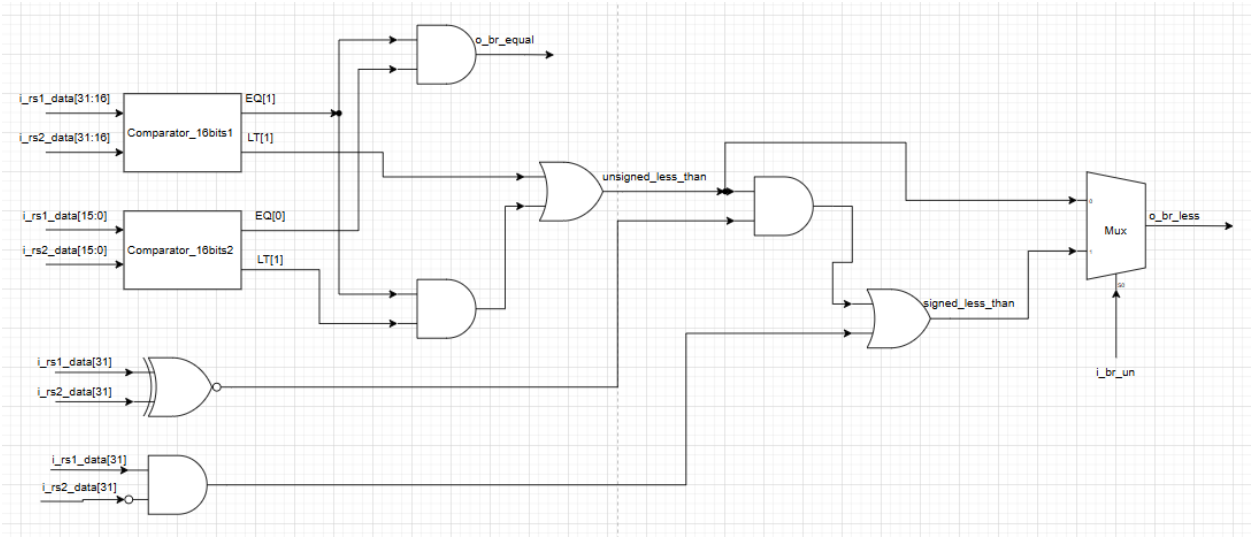
- Thiết kế bộ so sánh không dấu 2 số 8 bit: Tương tự khi vừa thiết kế bộ so sánh 4 bit.



- Thiết kế bộ so sánh không dấu 2 số 16 bit: Tương tự khi vừa thiết kế bộ so sánh 4 bit.



- Thiết kế toàn bộ BRC: Sử dụng hai bộ so sánh không dấu hai số 16bit vừa thiết kế:



o_br_equal = 1 khi A[31:16] = B[31:16] và A[15:0] = B[15:0]

Tín hiệu so sánh không dấu: unsigned_less_than = 1 khi i_rs1_data[31:16]<i_rs2_data[31:16] hoặc i_rs1_data [15:0] = i_rs2_data [15:0] và i_rs1_data [31:16] < i_rs2_data [31:16].

Tín hiệu so sánh có dấu: signed_less_than = 1 khi i_rs1_data[31] = 1 và i_rs2_data[31] = 0. Hoặc i_rs1_data[31] = i_rs2_data[31] và unsigned_less_than = 1.

Sau đó tín hiệu i_br_un sẽ chọn tín hiệu unsigned_less_than hay signed_less_than.

2.4. DMEM:

Bộ nhớ DMEM được sử dụng để lưu trữ data (các giá trị, các kết quả tính toán,…).

2.4.1. Specification

Module name: dmem.sv

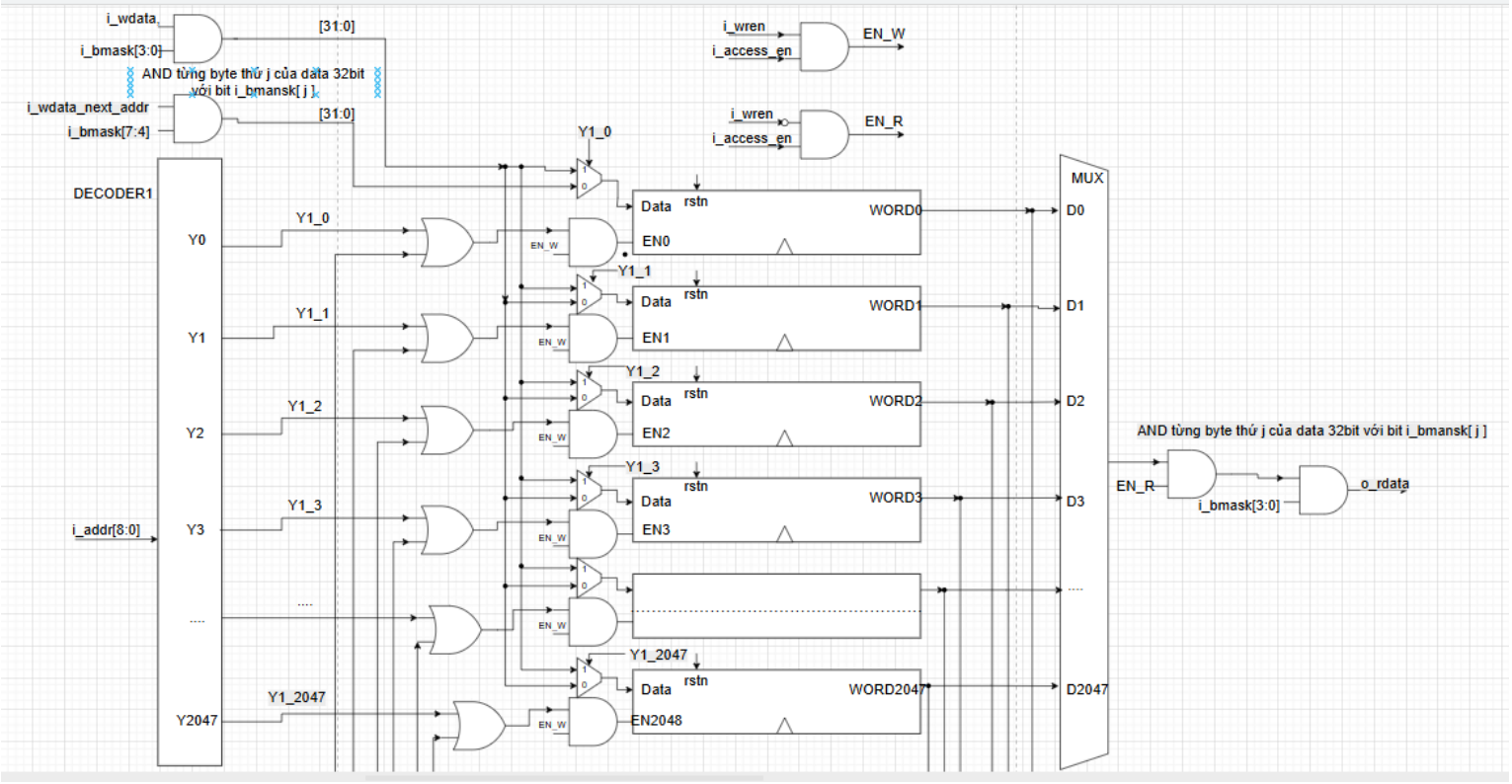
Tên Tín hiệu	Loại	Độ rộng	Mô tả
i_clk	Input	1 bit	Xung clock
i_rstn	Input	1 bit	Tín hiệu Reset mức thấp
i_access_en	Input	1 bit	Cho phép Truy cập (Chip Enable).- 1: Module hoạt động (Đọc hoặc Ghi).- 0: Module bị vô hiệu hóa, không ghi dữ liệu, đầu ra đọc trả về 0.

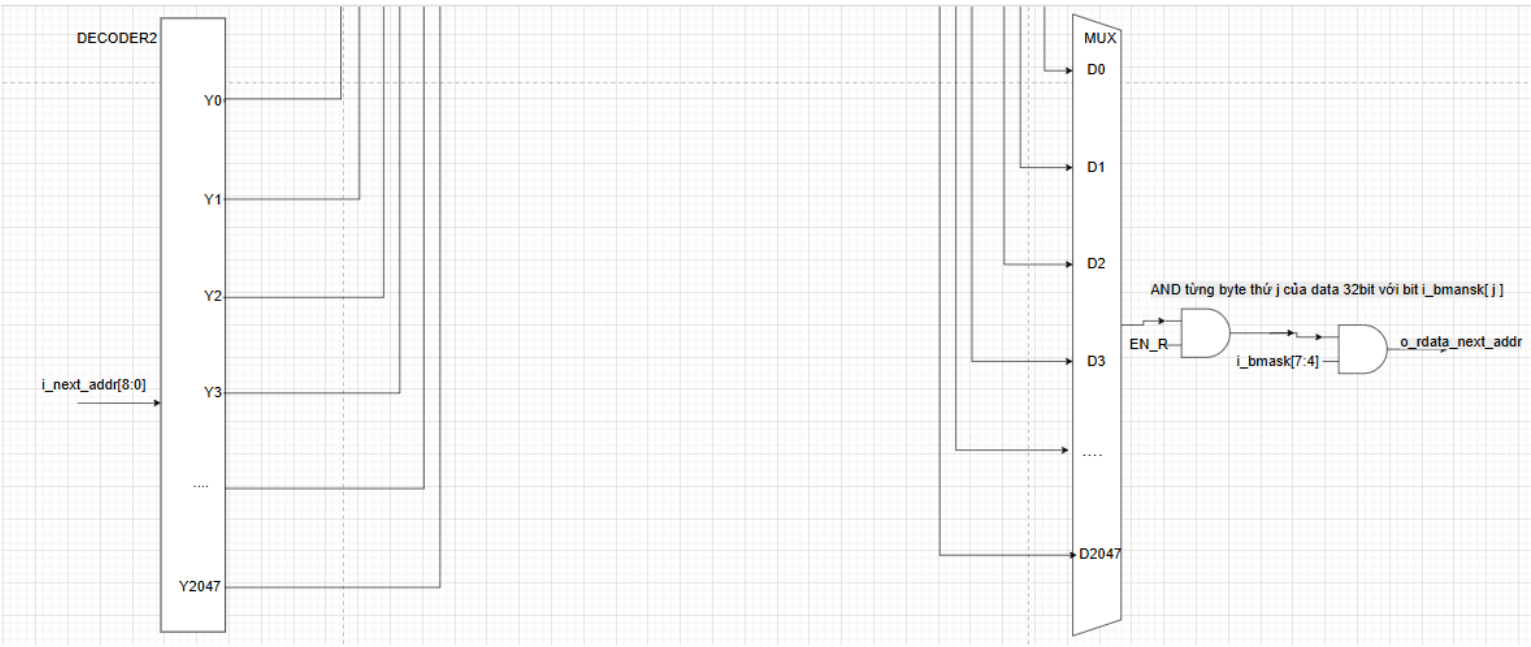
i_wren	Input	1 bit	Cho phép Ghi (Write Enable).- 1: Thực hiện thao tác Ghi (Write).- 0: Thực hiện thao tác Đọc (Read).
i_bmask	Input	8 bits	Byte Mask (Byte Enable). Kiểm soát việc đọc/ghi từng byte riêng lẻ cho cả hai cổng địa chỉ:- Bits [3:0]: Tương ứng với 4 byte tại i_addr.- Bits [7:4]: Tương ứng với 4 byte tại i_next_addr. Nếu bit là 1: Byte đó được ghi hoặc đọc. Nếu 0: Byte đó bị bỏ qua(đối với ghi) hoặc trả về 0 (đối với đọc).
i_addr	Input	9 bits	Địa chỉ truy cập chính. Trỏ đến từ nhớ (word) 32-bit cần truy cập.
i_wdata	Input	32 bits	Dữ liệu ghi cho cổng 1. Dữ liệu này sẽ được ghi vào địa chỉ i_addr (chỉ các byte được bật bởi i_bmask[3:0]).
o_rdata	Output	32 bits	Dữ liệu đọc từ cổng 1. Giá trị tại địa chỉ i_addr. Các byte không được chọn bởi i_bmask[3:0] sẽ trả về 0.
i_next_addr	Input	9 bits	Địa chỉ truy cập phụ (thường là i_addr + 1). Dùng để truy cập phần dữ liệu tràn sang word nhớ tiếp theo trong trường hợp xử lý dữ liệu misaligned.
i_wdata_next_addr	Input	32 bits	Dữ liệu này sẽ được ghi vào địa chỉ i_next_addr (chỉ các byte được bật bởi i_bmask[7:4]).
o_rdata_next_addr	Output	32 bits	Giá trị tại địa chỉ i_next_addr. Các byte không được chọn bởi i_bmask[7:4] sẽ trả về 0.

2.4.2. Thiết kế DMEM:

Ý tưởng của DMEM: Thiết kế bộ nhớ xử lý vấn đề misaligned. Sử dụng hai bộ decoder với hai đường địa chỉ là i_addr và i_next_addr. Điều này giúp truy cập đồng thời hai vị trí trong bộ nhớ. Để xử lý misaligned, i_next_addr = i_addr + 1, ta cần thêm một bộ cộng (nó sẽ được đặt trong module LSU sẽ phân tích sau). Các ngõ EN của các word được bật khi EN_W = 1 (tức i_wren = 1, cho phép ghi và i_access_en = 1, ngõ i_access_en là tín hiệu từ lsu để lựa chọn vùng nhớ được truy cập, sẽ được nói rõ hơn ở LSU) và phải có tín hiệu ngõ ra có vị trí tương tướng với vị trí của word được truy cập từ một trong hai bộ decoder. Do có hai ngõ vào data ghi là i_wdata và i_wdata_next_addr, nên một word có thể nhận một trong hai ngõ, ta cần đặt tại đầu vào word một mux2-1 để lựa chọn. Tín hiệu lựa chọn sẽ là các tín hiệu ngõ ra Y1_0, Y1_2, Y1_3,… Y1_2047 từ bộ decoder 1 tương ứng với ngõ vào i_addr. Tức là khi i_addr truy cập một ô word, ô word đó sẽ được ghi data từ i_wdata, ngược lại ta sẽ mặc định ghi i_wdata_next_addr. Do việc truy cập bộ nhớ không chỉ dừng ở việc truy cập word mà còn truy cập byte và half-word, ta cần các bit i_bmask[7:0] có nhiệm vụ lọc data nào được ghi vào word, i_bmask[3:0] lọc i_wdata, i_bmask[7:4] lọc i_wdata_next_addr. Bên cạnh việc ghi, ta cần xây dựng cấu hình đọc cho DMEM, ngõ EN_R = 1 tức cho phép đọc khi i_wren = 0 và i_access_en = 1. Bit này sẽ được AND với ngõ ra bộ của 2 MUX dùng để lựa chọn data đọc từ word với tín hiệu lựa chọn là

i_addr (cho MUX1) và và i_addr_next_addr (cho MUX2). Sau đó, dữ liệu cũng sẽ được lọc thông qua các bit i_bmask[7:0] để đưa ra ngõ ra o_rdata và o_rdata_next_addr.





2.5. Regfile (tương tự Dmem)

Tập thanh ghi (Regfile) là thành phần cốt lõi của CPU, đóng vai trò là bộ lưu trữ

dữ liệu ngắn hạn và nhanh hơn đáng kể so với các bộ nhớ khác. Trong kiến trúc RV32I, tập này chứa 32 thanh ghi đa năng (được đặt tên từ x0 đến x31), mỗi thanh ghi có độ rộng 32 bit.

2.5.1. Design ideal

Hoạt động của nó thông qua các ngõ đọc và ghi (tương tự DMEM):

Read Ports (Asynchronous): Regfile có cấu hình đọc kép (tức hai ngõ độc lập)

- Nó nhận hai đầu vào địa chỉ 5 bit, i_rs1_addr và i_rs2_addr, trong mã lệnh nhận từ IMEM.
- Ngõ đọc theo cấu trúc tổ hợp, tức bất đồng bộ.
- Nó xuất dữ liệu 32 bit trên o_rs1_data và o_rs2_data. Điều này có nghĩa là dữ liệu đọc gần như ngay lập tức thông qua các địa chỉ đọc nhận được, mà không cần chờ cạnh xung clock.

Write Port (Synchronous): Regfile có cấu hình ghi đơn (tức chỉ có một ngõ ghi).

- Nó nhận đầu vào địa chỉ 5 bit, i_rd_addr (địa chỉ ghi), đầu vào i_rd_data (dữ liệu ghi) 32 bit và tín hiệu i_rd_wren (cho phép ghi) 1 bit từ Control Unit (i_rd_wren = 1 là ghi, ngược lại là đọc).
- Ghi theo cấu hình đồng bộ: Ghi chỉ chỉ xảy ra ở cạnh clock.
- Nếu i_rd_wren =1, dữ liệu trên i_rd_data được ghi vào thanh ghi được chỉ định bởi địa chỉ i_rd_addr. Nếu i_rd_wren ở mức thấp, không có thanh ghi nào bị thay đổi.

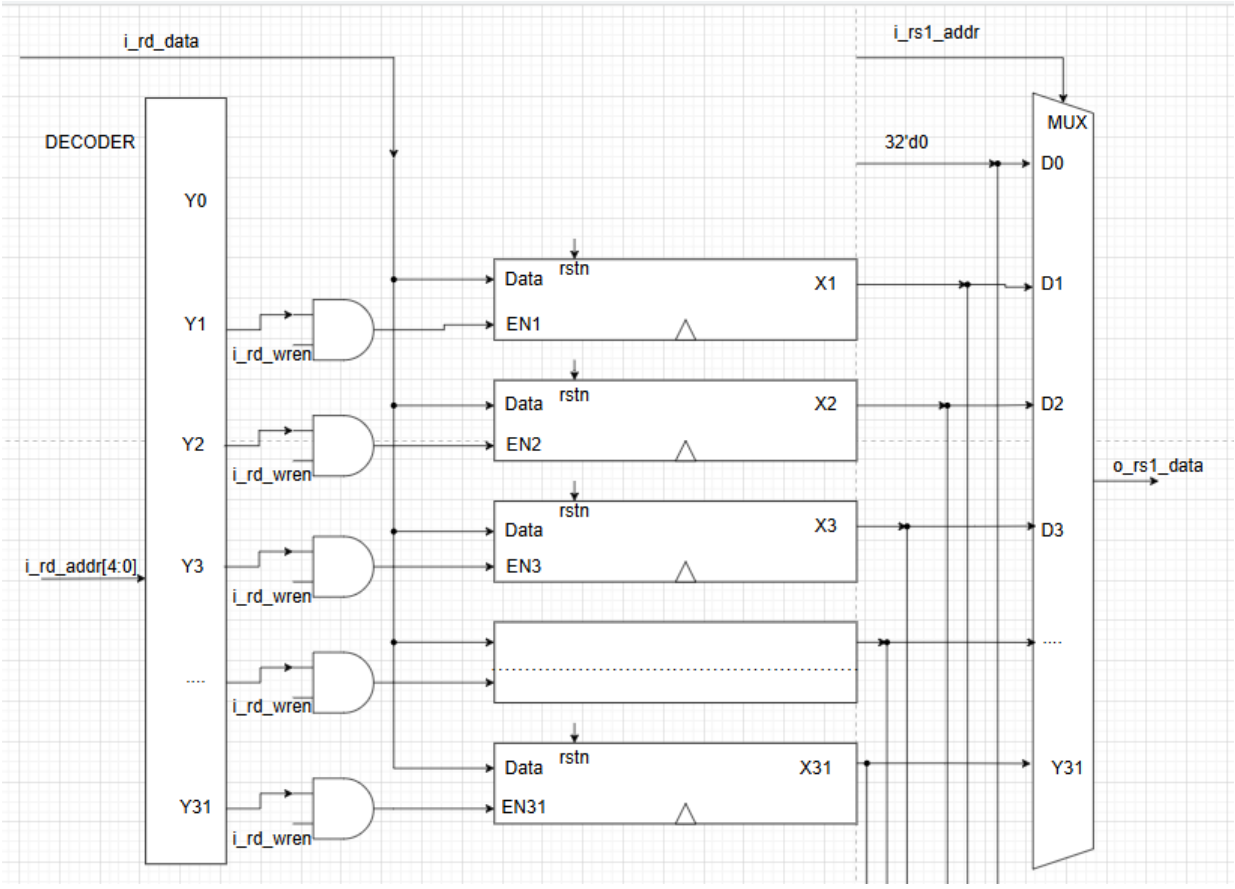
2.5.2. Specification

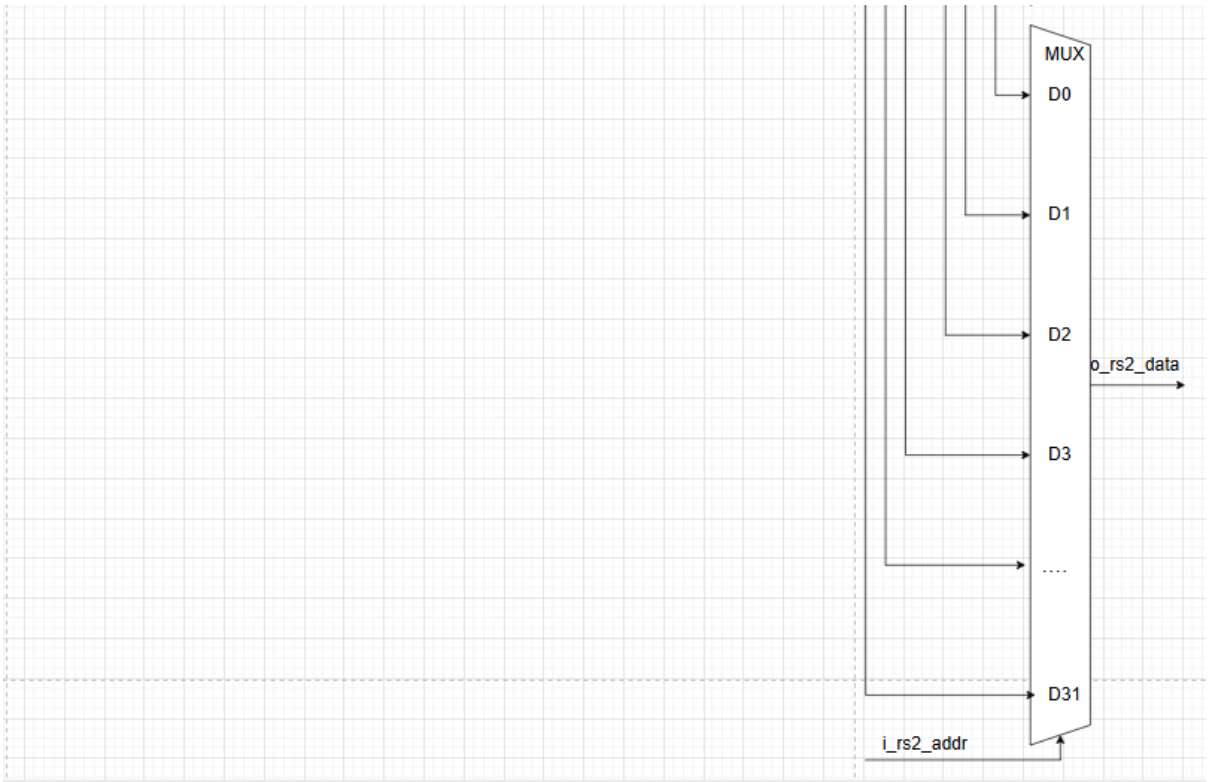
Module name: regfile.sv

Signal	Width	Direction	Description
i_clk	1	Input	Global clock, active on the rising edge.
i_rstn	1	Input	Global low active reset

i_rd_wren	1	Input	Register writes enable
i_rd_addr	5	Input	Destination registers address (0-31). Specifies which register to write to
i_rd_data	32	Input	Write data. The 32-bit data to be written into the destination register
i_rs1_addr	5	Input	Register 1 address (0-31). (Specifies the address for the first read port)
o_rs1_data	32	Output	Read data 1. (The 32-bit data read from the register at i_rs1_addr).
i_rs2_addr	5	Input	Register 2 address (0-31). (Specifies the address for the second read port)
o_rs2_data	32	Output	Read data 2. (The 32-bit data read from the register at i_rs2_addr)

2.5.3. Thiết kế Regfile: (Ý tưởng tương tự DMEM, tuy nhiên, ta chỉ cần một decoder để truy cập việc ghi và cũng sử dụng hai MUX cho hai ngõ ra đọc.


i_rd_data
i_rs1_addr
DECODER
MUX
32'd0
D0
Y0
Data
rstn
X1
D1
Y1
EN1
i_rd_wren
Data
rstn
X2
D2
Y2
EN2
i_rd_wren
o_rs1_data
rstn
Data
X3
D3
i_rd_addr[4:0]
Y3
EN3
i_rd_wren
....
....
i_rd_wren
Data
rstn
X31
Y31
Y31
EN31
i_rd_wren


MUX
D0
D1
D2
o_rs2_data
D3
....
D31
i_rs2_addr

2.6. Instruction Memory (IMEM)

IMEM (Bộ nhớ Lệnh) là một bộ nhớ dùng để lưu trữ và đọc các lệnh để thực thi ủa CPU.

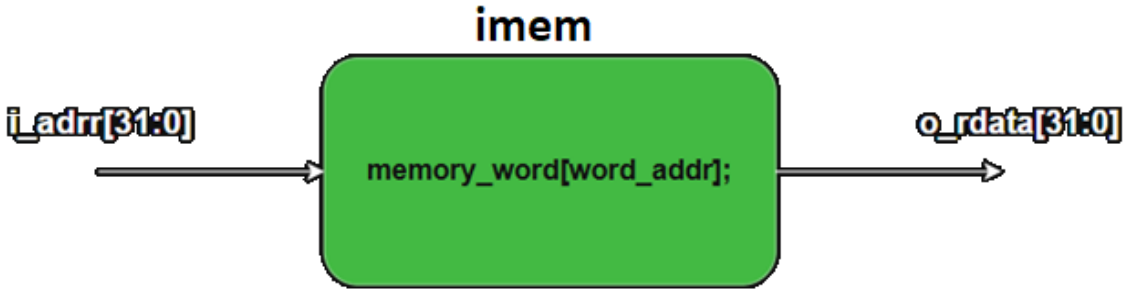
Trong thiết kế nhóm em, khối này hoạt động giống như một mạch tổ hợp lớn. Khi giá trị PC thay đổi theo mỗi chu kỳ xung nhịp, IMEM ngay lập tức (sau một khoảng trễ lan truyền) sẽ cung cấp lệnh tiếp theo cần thực thi.

2.6.1. Design ideas

Bộ nhớ IMEM sẽ nhận được địa chỉ đầu vào 32 bit, từ Bộ đếm Chương trình (PC). Dựa trên địa chỉ này, nó sẽ truy xuất lệnh 32 bit được lưu trữ tại địa chỉ tương ứng. Nó xuất lệnh 32 bit này ra datapath (đến các khối khác như Control Unit) để giải mã và thực thi.

Lưu ý rằng bộ nhớ IMEM này có dung lượng 8KiB.

Design sketch: Cấu hình các port của khối IMEM sẽ như hình dưới đây.



Cấu trúc của IMEM tương tự như Regfile nhưng được mở rộng với nhiều word nhớ hơn và cấu trúc load chỉ cần một MUX để có thể load lệnh tương ứng với địa chỉ nhận từ PC. Do i_addr có 32 bit, nên bộ nhớ có thể có lên tới 2^32 word nhớ.

2.6.2. Specification

Module name: imem.sv

Signal	Width	Direction	Description
i_addr	32	Input	Instruction address (the address received from the PC)
o_rdata	32	Output	Instruction read data. (The 32-bit

			instruction fetched from memory)

2.7. LSU

2.7.1. Specification

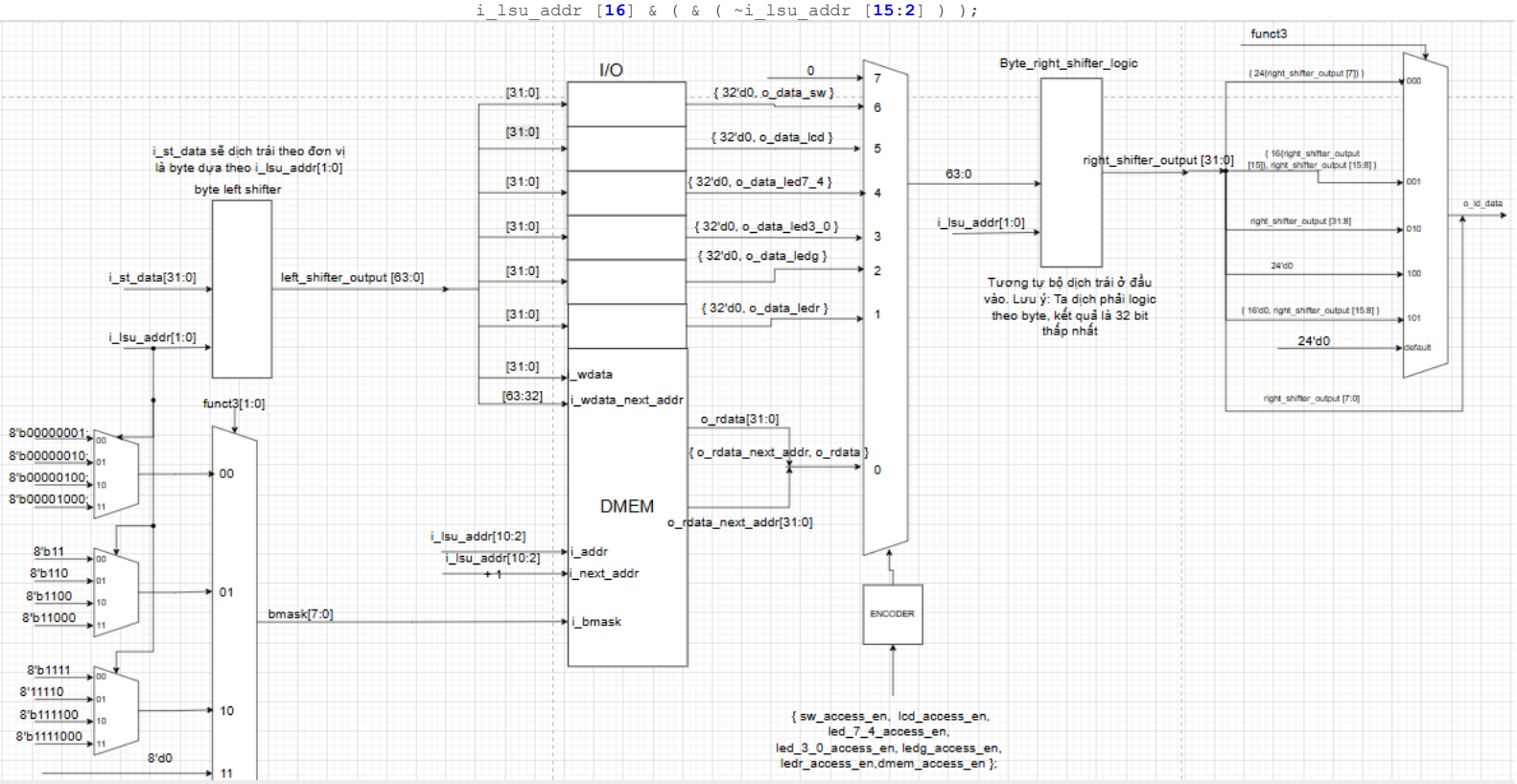
Module name: lsu.sv

Tên Cổng	Loại	Độ rộng (Width)	Mô tả
i_clk	Input	1 bit	Xung nhịp hệ thống (Clock).
i_reset	Input	1 bit	Tín hiệu Reset hệ thống.
i_lsu_addr	Input	32 bit	Địa chỉ truy cập bộ nhớ/ngoại vi. Được sử dụng để giải mã địa chỉ (dmem, led, lcd, v.v.) và tính toán byte shift.
i_st_data	Input	32 bit	Dữ liệu cần ghi (Store Data) vào bộ nhớ hoặc ngoại vi.
i_lsu_wren	Input	1 bit	Tín hiệu cho phép ghi (Write Enable). - 1: Ghi (Store). - 0: Đọc (Load).
funct3	Input	3 bit	Mã lệnh điều khiển kiểu truy cập (Byte/Half/Word) và kiểu mở rộng dấu (Signed/Unsigned). - [1:0]: 00 (Byte), 01 (Half), 10 (Word). - [2]: 1 (Unsigned), 0 (Signed).
i_io_sw	Input	32 bit	Dữ liệu đầu vào từ các Switches ngoại vi.
o_ld_data	Output	32 bit	Dữ liệu đọc ra (Load Data) từ bộ nhớ hoặc ngoại vi, đã được xử lý dịch bit và mở rộng dấu.
o_io_ledr	Output	32 bit	Dữ liệu điều khiển cho LED Đỏ (Red LEDs).
o_io_ledg	Output	32 bit	Dữ liệu điều khiển cho LED Xanh (Green LEDs).
o_io_hex0	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX0).
o_io_hex1	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX1).
o_io_hex2	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX2).
o_io_hex3	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX3).
o_io_hex4	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX4).
o_io_hex5	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX5).
o_io_hex6	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX6).
o_io_hex7	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX7).
o_io_lcd	Output	32 bit	Dữ liệu điều khiển cho màn hình LCD.

2.7.2. Ý tưởng thiết kế:

Đầu tiên, ta tạo các tín hiệu access là các minterm tổ hợp bởi các bit của i_lsu_addr[31:0]. Các tín hiệu access của các vùng I/O và vùng nhớ (DMEM). Các tín hiệu này cho phép truy cập với vùng I/O hay DMEM tương ứng:

```
//// enable access ////
logic dmem_access_en, ledr_access_en, ledg_access_en,
      led_3_0_access_en, led_7_4_access_en, lcd_access_en,  //// enable accessc for each memory domain ////
                        sw_access_en;
assign dmem_access_en     = & ( ~i_lsu_addr [31:11] );
assign ledr_access_en     = ( & ( ~i_lsu_addr [31:29] ) ) & i_lsu_addr [28] & ( & ( ~i_lsu_addr [27:2] ) );
assign ledg_access_en     = ( & ( ~i_lsu_addr [31:29] ) ) & i_lsu_addr [28] & ( & ( ~i_lsu_addr [27:13] ) ) &
                            i_lsu_addr [12] & ( & ( ~i_lsu_addr [11:2] ) );
assign led_3_0_access_en = ( & ( ~i_lsu_addr [31:29] ) ) & i_lsu_addr [28] & ( & ( ~i_lsu_addr [27:14] ) ) &
                            i_lsu_addr [13] & ( & ( ~i_lsu_addr [12:2] ) );
assign led_7_4_access_en = ( & ( ~i_lsu_addr [31:29] ) ) & i_lsu_addr [28] & ( & ( ~i_lsu_addr [27:14] ) ) &
                            i_lsu_addr [13] & i_lsu_addr [12] & ( & ( ~i_lsu_addr [11:2] ) );
assign lcd_access_en      = ( & ( ~i_lsu_addr [31:29] ) ) & i_lsu_addr [28] & ( & ( ~i_lsu_addr [27:15] ) ) &
                            i_lsu_addr [14] & ( & ( ~i_lsu_addr [13:2] ) );
assign sw_access_en       = ( & ( ~i_lsu_addr [31:29] ) ) & i_lsu_addr [28] & ( & ( ~i_lsu_addr [27:17] ) ) &
                            i_lsu_addr [16] & ( & ( ~i_lsu_addr [15:2] ) );
```



Dữ liệu từ i_st_data được đưa vào một bộ dịch trái theo byte dựa theo độ dịch là i_lsu_addr[1:0]. Bộ dịch trái có đầu là vào 32 bit và kết quả dịch có 64 bit. Điều này để xử lý vấn đề misaligned trong các địa chỉ không căn chỉnh như i_lsu_addr[1:0] = 01,10,11. Sau khi dịch, 32 bit đầu sẽ được đưa vào i_wdata, và 32 bit cao sẽ được đưa vào i_wdata_next_addr của bộ DMEM. Để có thể đáp ứng việc truy cập byte, half_word, ta cần tạo một bộ xử lý bmask dựa trên tín hiệu

i_lsu_addr[1:0] (đại diện cho địa chỉ trên toàn bộ 1 word nhớ của DMEM) và funct3 trong mã lệnh trong việc lựa chọn truy suất byte, half-word hay word. Kế tiếp, trong hoạt động load, ta cần một bộ MUX để lựa chọn vùng để xuất ngõ ra. Đây là MUX8-1, có 64 bộ tương ứng xử lý 64 bit. Đưa vào bộ MUX, ta cần ghép {o_rdata_next_addr, o_rd_data} của bộ DMEM. Kế tiếp, ta đưa 64 bit ngõ ra của MUX vào một bọ dịch phải cũng theo byte và dựa theo độ dịch là i_lsu_addr, đầu ra là 32 bit thấp nhất của kết quả phép dịch. Việc dịch phải này tương tự dịch trái, giúp xử lý misaligned. Sau khi đó, ta đưa 32 bit (right_shifter_output) xuất ra từ bộ dịch phải vào một bộ MUX mở rộng dấu cho kết quả load, ta xuất thẳng byte thấp nhất ra trước, do byte này không nằm trong việc mở rộng dấu. Dựa trên hàm funtc3[2:0] để lựa chọn việc mở rộng dấu tương ứng với lệnh. Nếu funtc3 = 000, thì mở rộng dấu cho lb, nếu 001 là mở rộng dấu cho lh, nếu 010 là lw nên ta đưa giữ nguyên toàn bộ 32bit cần load, nếu 100 là lbu nên không cần mở rộng dấu mà ta lấp đầy các bit cao bằng 0, nếu 101 là lhu cũng tương tự ý tưởng của lbu.

2.8. Immediate Generator

Immediate Generator là một khối logic tổ hợp chịu trách nhiệm giải mã và tạo ra giá trị tức thời - imm 32 bit, mở rộng dấu cho imm chính xác từ một từ lệnh 32 bit. Hoạt động của nó rất quan trọng để xử lý các lệnh sử dụng toán hạng imm.

2.8.1. Design ideal

Input: Khối này lấy từ lệnh 32 bit (cụ thể là bit [31:7]) làm đầu vào dữ liệu chính. Khối này cũng nhận tín hiệu điều khiển (i_ImmSel) từ Control Unit để chọn định dạng lệnh ứng với giá trị imm.

Function: Khối này về cơ bản là một bộ MUX lớn. Dựa trên tín hiệu điều khiển i_ImmSel, nó sẽ lựa chọn, sắp xếp lại và nối các bit thích hợp từ lệnh đến đầu ra imm 32 bit. Module này phải được thiết kế xuất kết quả chính xác cho tất cả các giá trị tức imm ứng với các Trường lệnh được quy định bởi RV32I ISA (kiểu I, kiểu S, kiểu B, kiểu U và kiểu J).

Mở rộng Dấu: Một chức năng quan trọng của khối này là mở rộng dấu. Khối này xác định bit dấu cho định dạng đã cho (hầu như luôn là inst[31]) và sao chép nó để lấp đầy tất cả các bit cao hơn của đầu ra 32 bit.

Output: Khối này tạo ra một giá trị tức thời 32 bit duy nhất, mở rộng dấu. Giá trị này sau đó thường được gửi làm một trong các đầu vào cho ALU (thông qua việc lựa chọn cho i_op_b của ALU).

2.8.2. Specification

Module name: immGen.sv

Signal	Width	Direction	Description
i_instr	32	Input	Instruction word received from imem
i_ImmSel	3	Input	Immediate select. (Control signal from ControlUnit select which format to decode
o_imm	32	Output	Sign-extended immediate value

Cấu trúc của immGen.sv là một bộ MUX8-1 với các ngõ vào được mô tả như code sau:

```
module immGen(
  input logic [31:0] i_instr,              // mã lệnh nhận được từ imem
  input  logic [2:0]  i_ImmSel,
  output logic [31:0] o_imm                           // giá trị imm tính được
);
// Mặc định luôn gán o_imm = 0 để tránh latch
logic [31:0] imm_val;                      // Ta sẽ dùng 1 tín hiệu trung gian
  always_comb begin
    case (i_ImmSel)
          3'b000: imm_val = {{20{i_instr[31]}}, i_instr[31:20]};//dạng I format, mở rộng bit dấu ra cho đủ 32
bit //

          3'b001: imm_val  = {{20{i_instr[31]}}, i_instr[31:20]};            //// load-I format ////

          3'b010: imm_val = {{20{i_instr[31]}}, i_instr[31:25], i_instr[11:7]};    //// S format: mở rộng bit
dấu và phải ghép bit  ////

          3'b011: imm_val= {{19{i_instr[31]}}, i_instr[31], i_instr[7],i_instr[30:25],i_instr[11:8], 1'b0};
//// B format ////
          3'b100: imm_val = {{11{i_instr[31]}},i_instr[31], i_instr[19:12],i_instr[20],i_instr[30:21],1'b0};
//// J format ////
          3'b101: imm_val = {i_instr [31:12], 12'b0};      //// lui-U format /////
```

```
        3'b110: imm_val = { i_instr [31:12], 12'b0 };      //// auipc-U format ////

        3'b111: imm_val = {{20{i_instr[31]}}, i_instr[31:20]};   //// jalr-I format ////

      default : imm_val = 32'd0;
    endcase
  end
  // Gán giá trị ra output
    assign o_imm = imm_val;
endmodule
```

2.9. Control Unit.

Bộ Điều khiển (Control Unit) về cơ bản là "bộ não" của CPU. Nhóm thiết kế nó theo dạng một khối logic tổ hợp điều khiển hoạt động của toàn bộ đường dẫn dữ liệu bằng cách tạo ra tất cả các tín hiệu điều khiển cần thiết.

Trong bộ xử lý chu kỳ đơn, tất cả các tín hiệu điều khiển này được tạo ra dựa tên mã bit lệnh và duy trì thời gian của một chu kỳ xung clock cho đến khi được thay đổi bởi mã lệnh kế tiếp.

2.9.1. Design ideal

Input: Đầu vào chính của nó là chính mã lệnh 32 bit.

Function: Nhiệm vụ chính của Bộ Điều khiển là giải mã lệnh. Nó đọc mã lệnh để xác định loại lệnh (ví dụ: loại R, loại I, Load, Store, Branch, Jump). Dựa trên nhận dạng này, nó tạo ra một tập hợp các tín hiệu đầu ra để điều khiển CPU tương ứng với lệnh.

Output: Các tín hiệu đầu ra này là các dòng nhị phân "bật/tắt" hoặc "chọn" để điều khiển các thành phần phần cứng khác.

2.9.2. Specification

Module name: ControlUnit.sv

Signal	Width	Direction	Description
i_instr	32	Input	Instruction word received from imem
i_br_equal	1	Input	Flag indicates rs1 = rs2 from BRC

i_br_less	1	Input	Flag indicates rs1 < rs2 from BRC
o_pc_sel	1	Output	Next PC source select. (1 if select PC+4 , 0 if select branch/jump target)
o_rd_wren	1	Output	Register file write enable. (Asserted to write data back to the Regfile).
o_br_un	1	Output	Branch unsigned select. (1 if signed, 0 if unsigned).
o_opa_sel	1	Output	ALU operand A select. (1 if select rs1_data, 0 if select PC).
o_opb_sel	1	Output	ALU Operand B select. (1 if select rs2_data, 0 if select immediate value from ImmGen).
o_alu_op	4	Output	ALU operation code. (Tells the ALU which operation to perform)
o_mem_wren	1	Output	Memory write enable. (0 if select alu result, 1 if select load data, and 2 if select PC +4)
o_wb_sel	2	Output	Write back data select. (Selects the data to write to the Regfile
o_ImmSel	3	Output	Immediate generator select. (Tells the ImmGen block which immediate format to decode).
o_insn_vld	1	Output	Instruction valid. (Indicates that the current instruction is valid).

2.10. Top module: Single Cycle

CPU syngle_cycle là module cấp cao nhất của toàn bộ thiết kế bộ xử lý. Nó không thực hiện một chức năng riêng lẻ; thay vào đó, nhiệm vụ của nó là khởi tạo và kết nối tất cả các mô-đun con riêng lẻ lại với nhau để tạo thành datapath RV32I hoàn chỉnh, hoạt động ổn định trong điều kiện không có ràng buộc về timing.

Module này cũng xử lý việc ghi hoặc đọc, và giao tiếp các thiết bị ngoại vi tiêu chuẩn như công tắc, đèn LED và màn hình LCD..

2.10.1. Design ideal

Chúng ta sẽ dựa vào Hình 1: CPU kết nối tất cả các khối đã thiết kế vào một khối cao nhất có tên là single_cycle.sv. Ngoài các khối đã thiết kế ở các phần trước, nhóm sẽ phải tạo một số mô-đun con khác như PC_reg.sv, 4-adder, MUX.

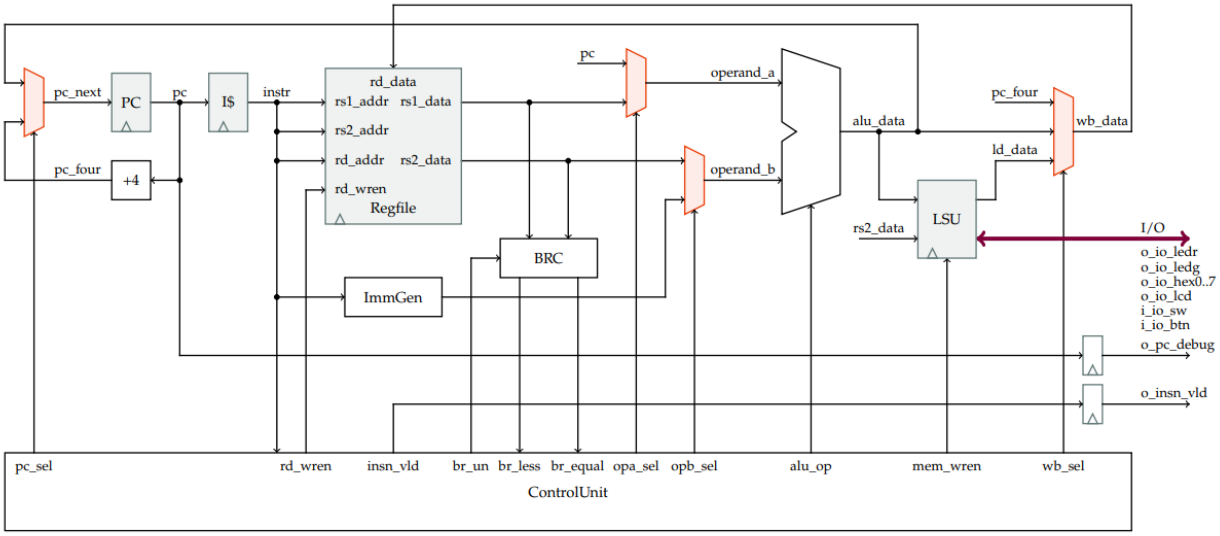


Figure 1: Single Cycle Processor

2.10.2. Specification

Module name: single_cycle.sv

Signal	Width	Direction	Description
i_clk	1	Input	Global clock, active on the rising edge.
i_reset	1	Input	Global low active reset

i_io_sw	32	Input	Input from the switches.
o_pc_debug	32	Output	Debug port to output the current Program Counter (PC) value
o_insn_vld	1	Output	Instruction valid flag. (Indicates a valid instruction)
o_io_ledr	32	Output	Output to control the 32 red LEDs.
o_io_ledg	32	Output	Output to control the 32 green LEDs
o_io_lcd	32	Output	Output for driving the LCD register.
o_io_hex0…7	7	Output	Output for driving 7-segment LED displays. (HEX0 to HEX7)

3. Verification Strategy

Trong phần này, nhóm sẽ sử dụng một chương trình testbench riêng để kiểm tra chức năng của các khối.

3.1. ALU

Code Testbench:

```
`timescale 1ns/1ps

module alu_tb ();
// stimulus input //
  logic [31:0] i_op_a, i_op_b;
  logic [3:0]  i_alu_op;

// expected output //
  logic [31:0] o_alu_data;

// design under test //
alu DUT (
  .i_op_a     ( i_op_a ),
  .i_op_b     ( i_op_b ),
  .i_alu_op   ( i_alu_op ),
  .o_alu_data ( o_alu_data )
);

// Definitions for verification logic
  logic [31:0] expected_result;
```

```
  // ALU Opcode Parameters (phải khớp với DUT)
  localparam OP_ADD    = 4'b0000;
  localparam OP_SUB    = 4'b0001;
  localparam OP_SLT    = 4'b0010;
  localparam OP_SLTU   = 4'b0011;
  localparam OP_XOR    = 4'b0100;
  localparam OP_OR     = 4'b0101;
  localparam OP_AND    = 4'b0110;
  localparam OP_SLL    = 4'b0111;
  localparam OP_SRL    = 4'b1000;
  localparam OP_SRA    = 4'b1001;
  localparam OP_PASS_B = 4'b1111;

//// stimulus simulator ////
int seed = 14159;

initial begin
  int count = 0;  //// the corect case counter ////
  $display ("----------------------------------------");
  $display (" Starting ALU Verification Loop ");
  $display ("----------------------------------------");

  // random stimulus loop //
  // Tăng số lần lặp lên để bao phủ đủ các trường hợp ngẫu nhiên của 11 phép toán
  for ( int i=1; i < 101; i++ ) begin
    i_op_a   = $random ( seed );
    i_op_b   = $random ( seed );
    // Random opcode trong khoảng hợp lệ (0-9 hoặc 15), dùng %16 để ra 4 bit
    i_alu_op = $random ( seed ) % 16;

    #5; // Chờ DUT xử lý

    // Tính toán kết quả mong đợi (Golden Model)
    case (i_alu_op)
        OP_ADD:    expected_result = i_op_a + i_op_b;
        OP_SUB:    expected_result = i_op_a - i_op_b;
        OP_SLT:    expected_result = ($signed(i_op_a) < $signed(i_op_b)) ? 32'd1 : 32'd0;
        OP_SLTU:   expected_result = (i_op_a < i_op_b) ? 32'd1 : 32'd0;
        OP_XOR:    expected_result = i_op_a ^ i_op_b;
        OP_OR:     expected_result = i_op_a | i_op_b;
        OP_AND:    expected_result = i_op_a & i_op_b;
        OP_SLL:    expected_result = i_op_a << i_op_b[4:0];
        OP_SRL:    expected_result = i_op_a >> i_op_b[4:0];
        OP_SRA:    expected_result = $signed(i_op_a) >>> i_op_b[4:0];
        OP_PASS_B: expected_result = i_op_b;
        default:   expected_result = 32'b0; // Các opcode không định nghĩa
    endcase

    // Kiểm tra kết quả
    if ( o_alu_data == expected_result ) begin
      count++;
      // In ít thông tin hơn để dễ nhìn, hoặc in chi tiết nếu muốn giống mẫu
```

```
      $display(" #Case %0d ||| Op: %b | A: %h | B: %h ||| Out: %h | Exp: %h ||| Correct", i,
i_alu_op, i_op_a, i_op_b, o_alu_data, expected_result);
    end
    else begin
      $display(" #Case %0d ||| Op: %b | A: %h | B: %h ||| Out: %h | Exp: %h ||| Error ", i,
i_alu_op, i_op_a, i_op_b, o_alu_data, expected_result);
    end
  end

  $display ( " Number of correct cases: %0d / 100 ", count );
  $finish;
end
endmodule : alu_tb
```

```
 #Case 78 ||| Op: 0100 | A: 5d887fbb | B: 52551ba4 ||| Out: 0fdd641f | Exp: 0fdd641f ||| Correct
 #Case 79 ||| Op: 1011 | A: d21ed6a4 | B: a86a6250 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 80 ||| Op: 0100 | A: 4ff64b9f | B: d291c6a5 ||| Out: 9d678d3a | Exp: 9d678d3a ||| Correct
 #Case 81 ||| Op: 0010 | A: cf69249e | B: 91940223 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 82 ||| Op: 1001 | A: 9b209c36 | B: 7b09c9f6 ||| Out: fffffe6c | Exp: fffffe6c ||| Correct
 #Case 83 ||| Op: 0100 | A: fd1f9efa | B: eabda8d5 ||| Out: 17a2362f | Exp: 17a2362f ||| Correct
 #Case 84 ||| Op: 0010 | A: f74d6cee | B: 61e3a1c3 ||| Out: 00000001 | Exp: 00000001 ||| Correct
 #Case 85 ||| Op: 0111 | A: 21bb8743 | B: 0de2151b ||| Out: 18000000 | Exp: 18000000 ||| Correct
 #Case 86 ||| Op: 1100 | A: 8e60901c | B: 78dafbf1 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 87 ||| Op: 0000 | A: 10e22b21 | B: 29dda353 ||| Out: 3abfce74 | Exp: 3abfce74 ||| Correct
 #Case 88 ||| Op: 0100 | A: 4047d980 | B: f5764aea ||| Out: b531936a | Exp: b531936a ||| Correct
 #Case 89 ||| Op: 0010 | A: 13bb8527 | B: d5ec6aab ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 90 ||| Op: 1101 | A: 64ee31c9 | B: 19577532 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 91 ||| Op: 1010 | A: 279bdd4f | B: 7c9f63f9 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 92 ||| Op: 0001 | A: ea3dc0d4 | B: 758f67eb ||| Out: 74ae58e9 | Exp: 74ae58e9 ||| Correct
 #Case 93 ||| Op: 1011 | A: 522865a4 | B: 39507d72 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 94 ||| Op: 0101 | A: 52790fa4 | B: 3e9cbb7d ||| Out: 7efdbffd | Exp: 7efdbffd ||| Correct
 #Case 95 ||| Op: 1101 | A: 72ded9e5 | B: 24ddc149 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 96 ||| Op: 0001 | A: 3ef31d7d | B: db7d58b6 ||| Out: 6375c4c7 | Exp: 6375c4c7 ||| Correct
 #Case 97 ||| Op: 0001 | A: 4ca82799 | B: 12337b24 ||| Out: 3a74ac75 | Exp: 3a74ac75 ||| Correct
 #Case 98 ||| Op: 0101 | A: 4cee8399 | B: 38fb1371 ||| Out: 7cff93f9 | Exp: 7cff93f9 ||| Correct
 #Case 99 ||| Op: 1100 | A: 2ede055d | B: d0ed4ca1 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 100 ||| Op: 1010 | A: 062b9d0c | B: c31b4686 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 Number of correct cases: 100 / 100
```

3.2. BRC

```
 `timescale 1ns/1ps
module tb ();
// stimulus input //
  logic [31:0] i_rs1_data, i_rs2_data;
  logic        i_br_un;
// expected output //
  logic        o_br_less;
  logic        o_br_equal;
// design under test //
brc_32bit DUT (
  .i_rs1_data ( i_rs1_data ),
  .i_rs2_data ( i_rs2_data ),
  .i_br_un    ( i_br_un ),
  .o_br_less  ( o_br_less ),
  .o_br_equal ( o_br_equal )
);
//// stimulus simulator ////
```

```
int seed = 14159;
initial begin
  int count = 0;  //// the corect case counter ////
  $display ("----------------------------------------");
  // random stimulus loop //
  for ( int i=0; i < 100; i++ ) begin
    i_rs1_data = $random ( seed );
    i_rs2_data = $random ( seed );
    i_br_un    = $random ( seed ); // L?y bit th?p nh?t làm giá tr? 0 ho?c 1
    #5
    // So sánh output c?a DUT v?i Logic chu?n (Golden Model)
    // Logic chu?n:
    // Less: N?u i_br_un=1 thì so sánh không d?u, ng??c l?i so sánh có d?u ($signed)
    // Equal: So sánh b?ng
    if ( { o_br_less, o_br_equal } == { (i_br_un ? (i_rs1_data < i_rs2_data) :
($signed(i_rs1_data) < $signed(i_rs2_data))), (i_rs1_data == i_rs2_data) } ) begin
      count++;
      $display(" #Case %0d ||| Simulus input: rs1 = %0d; rs2 = %0d; un = %0d ||| output: less=
%0d; equal = %0d ||| Correct ||| Number of correct cases: %0d  ", i, i_rs1_data, i_rs2_data,
i_br_un, o_br_less, o_br_equal, count);
    end
    else
      $display(" #Case %0d ||| Simulus input: rs1 = %0d; rs2 = %0d; un = %0d ||| output: less=
%0d; equal = %0d ||| Error ", i, i_rs1_data, i_rs2_data, i_br_un, o_br_less, o_br_equal);
  end
  $display ( " Number of correct cases: %0d / 100 ", count );
end
endmodule
```

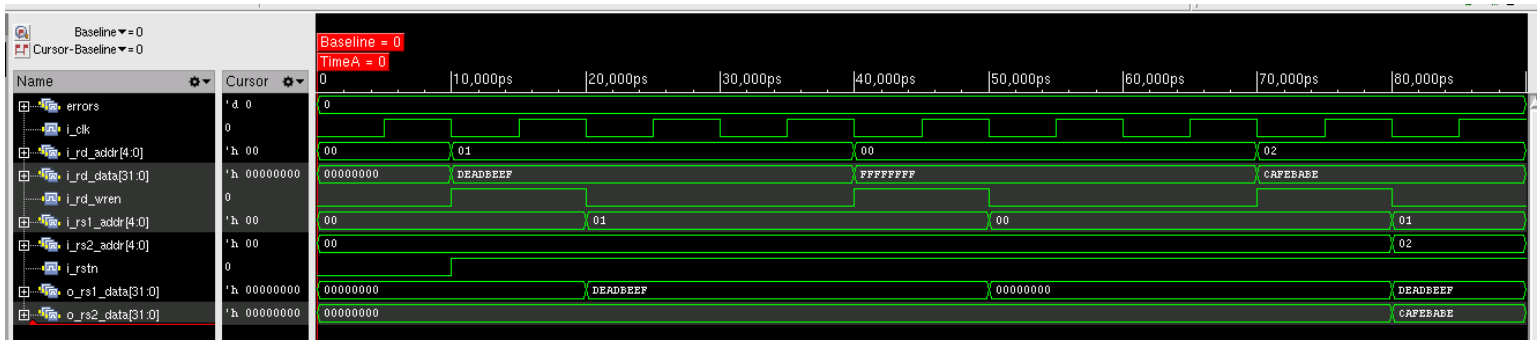
```
#  #Case 71 ||| Simulus input: rs1 = 4155757807; rs2 = 1324199325; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 72
#  #Case 72 ||| Simulus input: rs1 = 2243553291; rs2 = 1839571931; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 73
#  #Case 73 ||| Simulus input: rs1 = 1233659795; rs2 = 4040748769; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 74
#  #Case 74 ||| Simulus input: rs1 = 3699584185; rs2 = 1767931346; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 75
#  #Case 75 ||| Simulus input: rs1 = 3951165143; rs2 = 763352922; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 76
#  #Case 76 ||| Simulus input: rs1 = 696769875; rs2 = 47744261; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 77
#  #Case 77 ||| Simulus input: rs1 = 1569226683; rs2 = 1381309348; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 78
#  #Case 78 ||| Simulus input: rs1 = 3525236388; rs2 = 2825544272; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 79
#  #Case 79 ||| Simulus input: rs1 = 1341541279; rs2 = 3532768933; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 80
#  #Case 80 ||| Simulus input: rs1 = 3479774366; rs2 = 2442396195; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 81
#  #Case 81 ||| Simulus input: rs1 = 2602605622; rs2 = 2064239094; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 82
#  #Case 82 ||| Simulus input: rs1 = 4246707962; rs2 = 3938298069; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 83
#  #Case 83 ||| Simulus input: rs1 = 4149046510; rs2 = 1642308035; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 84
#  #Case 84 ||| Simulus input: rs1 = 565937987; rs2 = 232920347; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 85
#  #Case 85 ||| Simulus input: rs1 = 2388693020; rs2 = 2027617265; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 86
#  #Case 86 ||| Simulus input: rs1 = 283257633; rs2 = 702391123; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 87
#  #Case 87 ||| Simulus input: rs1 = 1078450560; rs2 = 4118170346; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 88
#  #Case 88 ||| Simulus input: rs1 = 331056423; rs2 = 3589040811; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 89
#  #Case 89 ||| Simulus input: rs1 = 1693331913; rs2 = 425162034; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 90
#  #Case 90 ||| Simulus input: rs1 = 664526159; rs2 = 2090820601; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 91
#  #Case 91 ||| Simulus input: rs1 = 3929915604; rs2 = 1972332523; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 92
#  #Case 92 ||| Simulus input: rs1 = 1378379172; rs2 = 961576306; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 93
#  #Case 93 ||| Simulus input: rs1 = 1383665572; rs2 = 1050459005; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 94
#  #Case 94 ||| Simulus input: rs1 = 1927207397; rs2 = 618512713; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 95
#  #Case 95 ||| Simulus input: rs1 = 1056120189; rs2 = 3682425014; un = 1 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 96
#  #Case 96 ||| Simulus input: rs1 = 1286088601; rs2 = 305363748; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 97
#  #Case 97 ||| Simulus input: rs1 = 1290699673; rs2 = 955978609; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 98
#  #Case 98 ||| Simulus input: rs1 = 786302301; rs2 = 3505212577; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 99
#  #Case 99 ||| Simulus input: rs1 = 103521548; rs2 = 3273344646; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 100
#  Number of correct cases: 100 / 100
```

3.3. REGFILE

```
`timescale 1ns/1ps

module tb();
```

```
  // 1. Khai báo tín hiệu kết nối với DUT
  logic        i_clk;
  logic        i_rstn;

  // Write Port
  logic        i_rd_wren;
  logic [4:0]  i_rd_addr;
  logic [31:0] i_rd_data;

  // Read Port 1
  logic [4:0]  i_rs1_addr;
  logic [31:0] o_rs1_data;

  // Read Port 2
  logic [4:0]  i_rs2_addr;
  logic [31:0] o_rs2_data;

  // 2. Khởi tạo DUT (Device Under Test)
  regfile DUT (
    .i_clk      ( i_clk ),
    .i_rstn     ( i_rstn ),
    .i_rd_wren  ( i_rd_wren ),
    .i_rd_addr  ( i_rd_addr ),
    .i_rd_data  ( i_rd_data ),
    .i_rs1_addr ( i_rs1_addr ),
    .o_rs1_data ( o_rs1_data ),
    .i_rs2_addr ( i_rs2_addr ),
    .o_rs2_data ( o_rs2_data )
  );
initial begin
  $dumpfile("tb.vcd");
  $dumpvars(1,tb );
end
  // 3. Tạo xung Clock (Chu kỳ 10ns)
    always #5 i_clk = ~i_clk;
  // 4. Biến đếm lỗi
  int errors = 0;

  // 5. Kịch bản kiểm tra (Test Scenario)
  initial begin
    // --- Khởi tạo ban đầu ---
    i_rstn      = 0;
    i_rd_wren   = 0;
    i_rd_addr   = 0;
    i_rd_data   = 0;
    i_rs1_addr  = 0;
    i_rs2_addr  = 0;
    i_clk =0 ;
    #10;
    // ============================================================
    // CASE 1: Ghi và Đọc chuẩn (Standard Write/Read Test)
    // ============================================================
    i_rstn = 1;
```

```
    // Ghi 0xDEADBEEF vào thanh ghi R1
    i_rd_wren = 1;
    i_rd_addr = 5'd1;
    i_rd_data = 32'hDEADBEEF;

    #10;
    i_rd_wren = 0;       // Tắt ghi

    i_rs1_addr = 5'd1;
    #10;


    // ============================================================
    // CASE 2: Bảo vệ ghi vào R0 (Write to R0 Protection Test)
    // ============================================================

    // Cố tình ghi giá trị khác 0 vào R0
    #10;
    i_rd_wren = 1;
    i_rd_addr = 5'd0;
    i_rd_data = 32'hFFFFFFFF; // Giá trị toàn 1


    #10;
    i_rd_wren = 0;

    // Đọc lại R0
    i_rs1_addr = 5'd0;
    #10;


    // ============================================================
    // CASE 3: Đọc đồng thời 2 cổng (Dual Port Read Test)
    // ============================================================

    // Chuẩn bị dữ liệu: Ghi vào R2 trước (R1 đã có dữ liệu từ Case 2)
    #10;
    i_rd_wren = 1;
    i_rd_addr = 5'd2;
    i_rd_data = 32'hCAFEBABE;

    #10;
    i_rd_wren = 0;

    // Thực hiện đọc đồng thời: Port 1 đọc R1, Port 2 đọc R2
    i_rs1_addr = 5'd1;
    i_rs2_addr = 5'd2;
    #10;
    $finish;
  end
endmodule
```



Case 1: Ta thực hiện ghi giá trị 32'hDEADBEEF vào địa chỉ x1, sau đó tiền hành đọc lại giá trị này cũng từ địa chỉ x1. Dựa trên dạng sóng ta thấy module hoạt động chính xác

Case 2: Ta cố tính ghi giá trị vào x0, dựa vào dạng sóng, x0 luôn load ra giá trị 32’d0.

Case 3: Ta ghi vào x2 giá trị 32'hCAFEBABE và sau đó cùng lúc đọc cả hai thanh ghi x1 và x2, dựa vào dạng sóng, ta thấy module hoạt động chính xác.

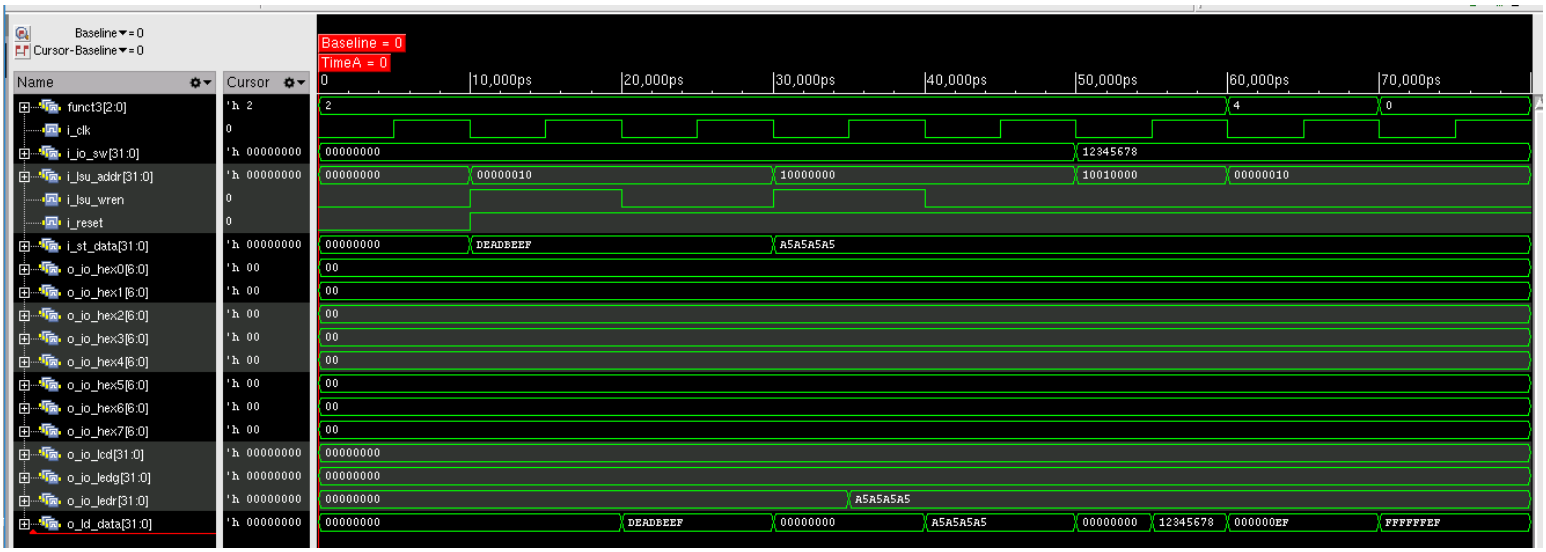
3.4. LSU

```
`timescale 1ns/1ps

module tb;

  // 1. Khai báo tín hiệu kết nối với DUT
  logic        i_clk;
  logic        i_reset;
  logic [31:0] i_lsu_addr;
  logic [31:0] i_st_data;
  logic        i_lsu_wren;
  logic [2:0]  funct3;
  logic [31:0] i_io_sw;

  logic [31:0] o_ld_data;
  logic [31:0] o_io_ledr;
  logic [31:0] o_io_ledg;
  logic [6:0]  o_io_hex0, o_io_hex1, o_io_hex2, o_io_hex3,
               o_io_hex4, o_io_hex5, o_io_hex6, o_io_hex7;
  logic [31:0] o_io_lcd;

  // 2. Khởi tạo DUT (Device Under Test)
  lsu DUT (
    .i_clk      ( i_clk ),
    .i_reset    ( i_reset ),
    .i_lsu_addr ( i_lsu_addr ),
    .i_st_data  ( i_st_data ),
```

```
    .i_lsu_wren ( i_lsu_wren ),
    .funct3     ( funct3 ),
    .i_io_sw    ( i_io_sw ),
    .o_ld_data  ( o_ld_data ),
    .o_io_ledr  ( o_io_ledr ),
    .o_io_ledg  ( o_io_ledg ),
    .o_io_hex0  ( o_io_hex0 ), .o_io_hex1 ( o_io_hex1 ),
    .o_io_hex2  ( o_io_hex2 ), .o_io_hex3 ( o_io_hex3 ),
    .o_io_hex4  ( o_io_hex4 ), .o_io_hex5 ( o_io_hex5 ),
    .o_io_hex6  ( o_io_hex6 ), .o_io_hex7 ( o_io_hex7 ),
    .o_io_lcd   ( o_io_lcd )
  );

  // 3. Tạo xung Clock (Chu kỳ 10ns)
  initial begin
    $dumpfile("tb.vcd");
    $dumpvars(0, tb );
  end
    always #5 i_clk = ~i_clk;

  // 4. Kịch bản kiểm tra (Test Scenario)
  initial begin
    // --- Khởi tạo ban đầu ---
    i_reset = 0; // Giả sử Reset active high (dựa trên logic bên trong dmem là ~rstn, nhưng
module lsu nhận i_reset)
                 // Cần kiểm tra kỹ: trong lsu, i_reset được nối vào i_rstn của dmem và io_reg.
                 // Thông thường reset active low là 0, active high là 1.
                 // Trong code dmem: if (~i_rstn) -> Reset khi i_rstn=0.
                 // Trong code lsu: .i_reset(i_reset). Vậy i_reset là active low.
    i_lsu_addr = 0;
    i_st_data  = 0;
    i_lsu_wren = 0;
    funct3     = 3'b010; // Word access
    i_io_sw    = 0;
    i_clk      =0;
    #10;
    // ============================================================
    // CASE 1: Ghi và Đọc DMEM (Word Access)
    // ============================================================
    // DMEM Base: 0x0000_0000

    // Ghi vào DMEM
    i_reset    = 1;
    i_lsu_addr = 32'h0000_0010; // Một địa chỉ trong vùng DMEM
    i_st_data  = 32'hDEADBEEF;
    i_lsu_wren = 1;             // Write
    funct3     = 3'b010;        // Word

    #10;
    i_lsu_wren = 0;   // Tắt ghi

    // Đọc lại
    #10;
```

```
    // ============================================================
    // CASE 2: Ghi và Đọc LEDR (I/O Access)
    // ============================================================
    // LEDR Base: 0x1000_0000

    i_lsu_addr = 32'h1000_0000;
    i_st_data  = 32'hA5A5A5A5;
    i_lsu_wren = 1;
    funct3     = 3'b010;

    #10;
    i_lsu_wren = 0;


    #10;


    // ============================================================
    // CASE 3: Đọc Switches (Input Peripheral)
    // ============================================================
    // Switches Base: 0x1001_0000

    // Giả lập tác động vào công tắc
    i_io_sw = 32'h12345678;

    i_lsu_addr = 32'h1001_0000;
    i_lsu_wren = 0; // Read
    funct3     = 3'b010;

    #10;

    // ============================================================
    // CASE 5: Byte Access (Load Byte Signed/Unsigned)
    // ============================================================
    // Sử dụng lại dữ liệu DEADBEEF tại 0x0000_0010
    // Byte 0 (LSB) là EF (1110_1111) -> Số âm nếu signed

    i_lsu_addr = 32'h0000_0010;
    i_lsu_wren = 0;

    // Test LBU (Load Byte Unsigned) - funct3 = 100
    funct3 = 3'b100;
    #10;
    // EF -> 000000EF
    // Test LB (Load Byte Signed) - funct3 = 000
    funct3 = 3'b000;
    #10;
    // EF -> FFFFFFEF (Sign extension)

    $finish;
end
```

```
endmodule
```



Biểu diễn qua 4 case:

Case 1: Ghi giá trị 32'hDEADBEEF vào địa chỉ 0x10 của dmem và đọc lại giá trị đó. Dựa trên waveform, ta có thể thấy case 1 hoạt động đúng.

Case 2: Ghi và đọc LEDR: Ghi 32'hA5A5A5A5 vào địa chỉ LEDR (0x10000000), và đọc lại giá trị đó, ta có thể thấy, wavform hoạt động đúng.

Case 3: Đọc SW: SW nhập vào 32'h12345678 và ta đọc được giá trị tương ứng trên waveform.

Case 4: Mở rộng dấu: Ta test trên các lệnh LBU và LB. Waveform hoạt động đúng.

3.5. CONTROl UNIT

```
`timescale 1ns/1ps

module tb();

  // 1. Khai báo tín hiệu kết nối với DUT
  logic [31:0] i_instr;
  logic        i_br_equal;
  logic        i_br_less;

  logic        o_pc_sel;
  logic        o_rd_wren;
  logic        o_insn_vld;
```

```
logic        o_br_un;
logic        o_opa_sel;
logic        o_opb_sel;
logic [3:0]  o_alu_op;
logic        o_mem_wren;
logic [1:0]  o_wb_sel;
logic [2:0]  o_ImmSel;

// 2. Khởi tạo DUT (Device Under Test)
ControlUnit DUT (
  .i_instr    ( i_instr ),
  .i_br_equal ( i_br_equal ),
  .i_br_less  ( i_br_less ),
  .o_pc_sel   ( o_pc_sel ),
  .o_rd_wren  ( o_rd_wren ),
  .o_insn_vld ( o_insn_vld ),
  .o_br_un    ( o_br_un ),
  .o_opa_sel  ( o_opa_sel ),
  .o_opb_sel  ( o_opb_sel ),
  .o_alu_op   ( o_alu_op ),
  .o_mem_wren ( o_mem_wren ),
  .o_wb_sel   ( o_wb_sel ),
  .o_ImmSel   ( o_ImmSel )
);

// 3. Các biến cho Golden Model (Kết quả mong đợi)
logic        exp_pc_sel;
logic        exp_rd_wren;
logic        exp_insn_vld;
logic        exp_br_un;
logic        exp_opa_sel;
logic        exp_opb_sel;
logic [3:0]  exp_alu_op;
logic        exp_mem_wren;
logic [1:0]  exp_wb_sel;
logic [2:0]  exp_ImmSel;

// Các trường trích xuất từ instruction để tính toán
logic [6:0] opcode;
logic [2:0] funct3;
logic       funct7b5;

// ALU Opcode Parameters (Phải khớp với DUT)
localparam OP_ADD    = 4'b0000;
localparam OP_SUB    = 4'b0001;
localparam OP_SLT    = 4'b0010;
localparam OP_SLTU   = 4'b0011;
localparam OP_XOR    = 4'b0100;
localparam OP_OR     = 4'b0101;
localparam OP_AND    = 4'b0110;
localparam OP_SLL    = 4'b0111;
localparam OP_SRL    = 4'b1000;
localparam OP_SRA    = 4'b1001;
```

```
localparam OP_PASS_B = 4'b1111;

// Opcodes
localparam OPCODE_R_TYPE = 7'b0110011;
localparam OPCODE_I_TYPE = 7'b0010011;
localparam OPCODE_LOAD   = 7'b0000011;
localparam OPCODE_STORE  = 7'b0100011;
localparam OPCODE_BRANCH = 7'b1100011;
localparam OPCODE_JALR   = 7'b1100111;
localparam OPCODE_JAL    = 7'b1101111;
localparam OPCODE_AUIPC  = 7'b0010111;
localparam OPCODE_LUI    = 7'b0110111;

//// stimulus simulator ////
int seed = 14159;

// Task để tạo Instruction ngẫu nhiên hợp lệ
task generate_random_instr;
  logic [6:0] rand_opcode;
  logic [2:0] rand_funct3;
  logic       rand_funct7b5;
  int         type_selector;

  type_selector = $random(seed) % 9; // Chọn 1 trong 9 loại lệnh

  case(type_selector)
    0: rand_opcode = OPCODE_R_TYPE;
    1: rand_opcode = OPCODE_I_TYPE;
    2: rand_opcode = OPCODE_LOAD;
    3: rand_opcode = OPCODE_STORE;
    4: rand_opcode = OPCODE_BRANCH;
    5: rand_opcode = OPCODE_JALR;
    6: rand_opcode = OPCODE_JAL;
    7: rand_opcode = OPCODE_AUIPC;
    8: rand_opcode = OPCODE_LUI;
    default: rand_opcode = OPCODE_R_TYPE;
  endcase

  rand_funct3   = $random(seed);
  rand_funct7b5 = $random(seed);

  // Ghép thành lệnh 32-bit (các phần khác random)
  i_instr = {1'b0, rand_funct7b5, 5'b0, 5'b0, 5'b0, rand_funct3, 5'b0, rand_opcode};
  // Lưu ý: Chỉ quan tâm opcode, funct3, funct7[5] cho Control Unit
  // Các trường rs1, rs2, rd, imm có thể random nhưng không ảnh hưởng logic này
  i_instr[31:25] = {1'b0, rand_funct7b5, 5'b0}; // Set funct7
  i_instr[24:20] = $random(seed); // rs2
  i_instr[19:15] = $random(seed); // rs1
  i_instr[11:7]  = $random(seed); // rd
endtask


initial begin
```

```
    int count = 0;  //// the corect case counter ////
    $display ("----------------------------------------");
    $display (" Starting ControlUnit Verification Loop ");
    $display ("----------------------------------------");

    // random stimulus loop //
    for ( int i=0; i < 100; i++ ) begin

      // 1. Tạo Stimulus
      generate_random_instr();
      i_br_equal = $random(seed);
      i_br_less  = $random(seed);

      #5; // Chờ DUT xử lý

      // 2. Tính toán kết quả mong đợi (Golden Model - Logic y hệt DUT)
      opcode   = i_instr[6:0];
      funct3   = i_instr[14:12];
      funct7b5 = i_instr[30];

      // Reset expected values
      exp_pc_sel = 0; exp_rd_wren = 0; exp_insn_vld = 0; exp_br_un = 0;
      exp_opa_sel = 0; exp_opb_sel = 0; exp_mem_wren = 0; exp_wb_sel = 0;
      exp_ImmSel = 0; exp_alu_op = OP_ADD;

      // Logic giải mã
      case (opcode)
        OPCODE_R_TYPE: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opb_sel = 0; exp_wb_sel = 0;
          case (funct3)
            3'b000: exp_alu_op = (funct7b5) ? OP_SUB : OP_ADD;
            3'b001: exp_alu_op = OP_SLL;
            3'b010: exp_alu_op = OP_SLT;
            3'b011: exp_alu_op = OP_SLTU;
            3'b100: exp_alu_op = OP_XOR;
            3'b101: exp_alu_op = (funct7b5) ? OP_SRA : OP_SRL;
            3'b110: exp_alu_op = OP_OR;
            3'b111: exp_alu_op = OP_AND;
            default: exp_alu_op = OP_ADD;
          endcase
        end
        OPCODE_I_TYPE: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opb_sel = 1; exp_wb_sel = 0; exp_ImmSel =
3'b000;
          case (funct3)
            3'b000: exp_alu_op = OP_ADD;
            3'b010: exp_alu_op = OP_SLT;
            3'b011: exp_alu_op = OP_SLTU;
            3'b100: exp_alu_op = OP_XOR;
            3'b110: exp_alu_op = OP_OR;
            3'b111: exp_alu_op = OP_AND;
            3'b001: exp_alu_op = OP_SLL;
            3'b101: exp_alu_op = (funct7b5) ? OP_SRA : OP_SRL;
```

```
            default: exp_alu_op = OP_ADD;
          endcase
        end
        OPCODE_LOAD: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opb_sel = 1; exp_wb_sel = 1; exp_ImmSel =
3'b001;
          exp_alu_op = OP_ADD;
        end
        OPCODE_STORE: begin
          exp_mem_wren = 1; exp_insn_vld = 1; exp_opb_sel = 1; exp_ImmSel = 3'b010;
          exp_alu_op = OP_ADD;
        end
        OPCODE_BRANCH: begin
          exp_insn_vld = 1; exp_opa_sel = 1; exp_opb_sel = 1; exp_wb_sel = 0; exp_ImmSel =
3'b011;
          exp_br_un = funct3[1]; // Bit 13 is bit 1 of funct3
          exp_alu_op = OP_ADD;
          // Branch Logic
          case (funct3)
            3'b000: exp_pc_sel = i_br_equal;
            3'b001: exp_pc_sel = !i_br_equal;
            3'b100: exp_pc_sel = i_br_less;
            3'b101: exp_pc_sel = i_br_equal | !i_br_less;
            3'b110: exp_pc_sel = i_br_less;
            3'b111: exp_pc_sel = i_br_equal | !i_br_less;
            default: exp_pc_sel = 0;
          endcase
        end
        OPCODE_JAL: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opa_sel = 1; exp_opb_sel = 1; exp_wb_sel = 2;
exp_ImmSel = 3'b100;
          exp_pc_sel = 1; exp_alu_op = OP_ADD;
        end
        OPCODE_JALR: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opb_sel = 1; exp_wb_sel = 2; exp_ImmSel =
3'b111;
          exp_pc_sel = 1; exp_alu_op = OP_ADD;
        end
        OPCODE_LUI: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opb_sel = 1; exp_wb_sel = 0; exp_ImmSel =
3'b101;
          exp_alu_op = OP_PASS_B;
        end
        OPCODE_AUIPC: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opa_sel = 1; exp_opb_sel = 1; exp_wb_sel = 0;
exp_ImmSel = 3'b110;
          exp_alu_op = OP_ADD;
        end
        default: ;
      endcase


      // 3. So sánh kết quả
```

```
      if ( {o_pc_sel, o_rd_wren, o_insn_vld, o_br_un, o_opa_sel, o_opb_sel, o_alu_op, o_mem_wren,
o_wb_sel, o_ImmSel} ==
           {exp_pc_sel, exp_rd_wren, exp_insn_vld, exp_br_un, exp_opa_sel, exp_opb_sel,
exp_alu_op, exp_mem_wren, exp_wb_sel, exp_ImmSel} )
      begin
        count++;
        $display(" #Case %0d ||| Opcode: %b | ALU_OP: %b ||| Correct", i, opcode, o_alu_op);
         $display("     Expected: PC=%b RD=%b VLD=%b BrU=%b OpA=%b OpB=%b ALU=%b Mem=%b WB=%b
Imm=%b",
                  exp_pc_sel, exp_rd_wren, exp_insn_vld, exp_br_un, exp_opa_sel, exp_opb_sel,
exp_alu_op, exp_mem_wren, exp_wb_sel, exp_ImmSel);
        $display("     Actual:   PC=%b RD=%b VLD=%b BrU=%b OpA=%b OpB=%b ALU=%b Mem=%b WB=%b
Imm=%b",
                  o_pc_sel, o_rd_wren, o_insn_vld, o_br_un, o_opa_sel, o_opb_sel, o_alu_op,
o_mem_wren, o_wb_sel, o_ImmSel);

      end
      else begin
        $display(" #Case %0d ||| Opcode: %b | F3: %b | F7: %b ||| ERROR", i, opcode, funct3,
funct7b5);
        $display("     Expected: PC=%b RD=%b VLD=%b BrU=%b OpA=%b OpB=%b ALU=%b Mem=%b WB=%b
Imm=%b",
                  exp_pc_sel, exp_rd_wren, exp_insn_vld, exp_br_un, exp_opa_sel, exp_opb_sel,
exp_alu_op, exp_mem_wren, exp_wb_sel, exp_ImmSel);
        $display("     Actual:   PC=%b RD=%b VLD=%b BrU=%b OpA=%b OpB=%b ALU=%b Mem=%b WB=%b
Imm=%b",
                  o_pc_sel, o_rd_wren, o_insn_vld, o_br_un, o_opa_sel, o_opb_sel, o_alu_op,
o_mem_wren, o_wb_sel, o_ImmSel);
      end
    end

    $display ( " Number of correct cases: %0d / 100 ", count );
    $finish;
  end

endmodule
```

```
#       Actual:   PC=1 RD=1 VLD=1 BrU=0 OpA=1 OpB=1 ALU=0000 Mem=0 WB=10 Imm=100
#   #Case 92 ||| Opcode: 0110011 | ALU_OP: 0011 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0011 Mem=0 WB=00 Imm=000
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0011 Mem=0 WB=00 Imm=000
#   #Case 93 ||| Opcode: 0110011 | ALU_OP: 0011 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0011 Mem=0 WB=00 Imm=000
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0011 Mem=0 WB=00 Imm=000
#   #Case 94 ||| Opcode: 0000011 | ALU_OP: 0000 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=1 ALU=0000 Mem=0 WB=01 Imm=001
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=1 ALU=0000 Mem=0 WB=01 Imm=001
#   #Case 95 ||| Opcode: 0110011 | ALU_OP: 0100 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0100 Mem=0 WB=00 Imm=000
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0100 Mem=0 WB=00 Imm=000
#   #Case 96 ||| Opcode: 0100011 | ALU_OP: 0000 ||| Correct
#       Expected: PC=0 RD=0 VLD=1 BrU=0 OpA=0 OpB=1 ALU=0000 Mem=1 WB=00 Imm=010
#       Actual:   PC=0 RD=0 VLD=1 BrU=0 OpA=0 OpB=1 ALU=0000 Mem=1 WB=00 Imm=010
#   #Case 97 ||| Opcode: 0110011 | ALU_OP: 0100 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0100 Mem=0 WB=00 Imm=000
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0100 Mem=0 WB=00 Imm=000
#   #Case 98 ||| Opcode: 0010111 | ALU_OP: 0000 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=1 OpB=1 ALU=0000 Mem=0 WB=00 Imm=110
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=1 OpB=1 ALU=0000 Mem=0 WB=00 Imm=110
#   #Case 99 ||| Opcode: 0110011 | ALU_OP: 0011 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0011 Mem=0 WB=00 Imm=000
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0011 Mem=0 WB=00 Imm=000
#   Number of correct cases: 100 / 100
```

3.6. IMMEDIATE GENERATION

```
`timescale 1ns/1ps

module tb();

  // 1. Khai báo tín hi?u k?t n?i v?i DUT
  logic [31:0] i_instr;
  logic [2:0]  i_ImmSel;
  logic [31:0] o_imm;

  // 2. Kh?i t?o DUT (Device Under Test)
  immGen DUT (
    .i_instr  ( i_instr ),
    .i_ImmSel ( i_ImmSel ),
    .o_imm    ( o_imm )
  );

  // 3. Bi?n cho Golden Model (K?t qu? mong ??i)
  logic [31:0] exp_imm;

  // ??nh ngh?a các h?ng s? cho ImmSel ?? code d? ??c (tùy ch?n)
  localparam IMM_I     = 3'b000;
  localparam IMM_LOAD  = 3'b001;
  localparam IMM_S     = 3'b010;
  localparam IMM_B     = 3'b011;
  localparam IMM_J     = 3'b100;
  localparam IMM_LUI   = 3'b101;
```

```
  localparam IMM_AUIPC = 3'b110;
  localparam IMM_JALR  = 3'b111;

  //// Stimulus simulator ////
  int seed = 14159;

  initial begin
    int count = 0;  //// Bi?n ??m s? case ?úng ////
    $display ("----------------------------------------");
    $display (" Starting ImmGen Verification Loop ");
    $display ("----------------------------------------");

    // Vòng l?p t?o stimulus ng?u nhiên
    for ( int i = 0; i < 100; i++ ) begin

      // 1. T?o Input Ng?u nhiên
      i_instr  = $random(seed);      // T?o l?nh 32-bit ng?u nhiên
      i_ImmSel = $random(seed) % 8;  // T?o ImmSel ng?u nhiên t? 0-7

      #5; // Ch? DUT x? lý (logic t? h?p nên r?t nhanh)

      // 2. Tính toán k?t qu? mong ??i (Golden Model)
      // Sao chép logic gi?i mã t? thi?t k? ?? ki?m tra
      case (i_ImmSel)
        IMM_I:     exp_imm = {{20{i_instr[31]}}, i_instr[31:20]};
        IMM_LOAD:  exp_imm = {{20{i_instr[31]}}, i_instr[31:20]};
        IMM_S:     exp_imm = {{20{i_instr[31]}}, i_instr[31:25], i_instr[11:7]};
        // B-type: Bit 0 luôn là 0, các bit khác ???c s?p x?p l?i t? instruction
        IMM_B:     exp_imm = {{19{i_instr[31]}}, i_instr[31], i_instr[7], i_instr[30:25],
i_instr[11:8], 1'b0};
        // J-type: Bit 0 luôn là 0
        IMM_J:     exp_imm = {{11{i_instr[31]}}, i_instr[31], i_instr[19:12], i_instr[20],
i_instr[30:21], 1'b0};
        IMM_LUI:   exp_imm = {i_instr[31:12], 12'b0};
        IMM_AUIPC: exp_imm = {i_instr[31:12], 12'b0};
        IMM_JALR:  exp_imm = {{20{i_instr[31]}}, i_instr[31:20]};
        default:   exp_imm = 32'd0;
      endcase

      // 3. So sánh k?t qu?
      if (o_imm == exp_imm) begin
        count++;
        // Có th? uncomment dòng d??i n?u mu?n xem chi ti?t các case ?úng
         $display(" #Case %0d ||| Sel: %b | Instr: %h ||| Correct: %h", i, i_ImmSel, i_instr,
o_imm);
        $display("     Expected: %h", exp_imm);
        $display("     Actual:   %h", o_imm);
      end else begin
        $display(" #Case %0d ||| Sel: %b | Instr: %h ||| ERROR", i, i_ImmSel, i_instr);
        $display("     Expected: %h", exp_imm);
        $display("     Actual:   %h", o_imm);
      end
```

```
    end

    $display ( " Number of correct cases: %0d / 100 ", count );
    $finish;
  end

endmodule
```

```
#       Expected: 00000016
#       Actual:   00000016
#  #Case 92 ||| Sel: 010 | Instr: e61c86cc ||| Correct: fffffe6d
#       Expected: fffffe6d
#       Actual:   fffffe6d
#  #Case 93 ||| Sel: 011 | Instr: c34fc086 ||| Correct: fffffc20
#       Expected: fffffc20
#       Actual:   fffffc20
#  #Case 94 ||| Sel: 110 | Instr: 507409a0 ||| Correct: 50740000
#       Expected: 50740000
#       Actual:   50740000
#  #Case 95 ||| Sel: 110 | Instr: a0b3de41 ||| Correct: a0b3d000
#       Expected: a0b3d000
#       Actual:   a0b3d000
#  #Case 96 ||| Sel: 101 | Instr: 30a2af61 ||| Correct: 30a2a000
#       Expected: 30a2a000
#       Actual:   30a2a000
#  #Case 97 ||| Sel: 011 | Instr: db69b0b6 ||| Correct: fffffda0
#       Expected: fffffda0
#       Actual:   fffffda0
#  #Case 98 ||| Sel: 110 | Instr: ce37c89c ||| Correct: ce37c000
#       Expected: ce37c000
#       Actual:   ce37c000
#  #Case 99 ||| Sel: 011 | Instr: 166c652c ||| Correct: 0000016a
#       Expected: 0000016a
#       Actual:   0000016a
#  Number of correct cases: 100 / 100
```

4. Alternative Design.

5. Evaluation

5.1. Achievements

Nhìn chung, nhóm chúng em đã hoàn thành milestone 2 tương đối ổn và đúng hạn. Khối CPU đã pass qua tất cả các bài kiểm tra trên server.

Ngoài ra, có thể chạy các chương trình ứng dụng từ dễ đến khó để kiểm tra các thiết bị ngoại vi trên bộ kit fpga DE2.

Từ cột mốc 2 này, chúng em đã có được một phần nào đó kiến thức căn bản về kiến trúc RISC-V, cách xây dựng bộ xử lý chu kỳ đơn, cách sử dụng máy chủ và

các công cụ liên quan. Ngoài ra, các em đã ôn tập lại kiến thức về các môn học cũ như Vi xử lý và Thiết kế Logic Số.

5.2. Obstacles

- Bên cạnh những thuận lợi, nhóm cũng gặp một số khó khăn khi làm việc, ví dụ:
- Gặp khó khăn trong việc thiết kế, đặc biệt là khối lsu.sv.
- Gặp khó khăn trong việc triển khai CPU trên kit FPGA và mã assembly trên CPU để hiển thị trên LCD 16x2. Đặc biệt là các bộ nhớ như DMEM, IMEM đã được sử dụng trên quá nhiều phần tử logic trên FPGA, dẫn đến tình trạng tràn phần tử FPGA. Giải pháp trong trường hợp này là nhóm đã giảm số lượng word trong DMEM và IMEM để phù hợp với ứng dụng của nhóm.
- Gặp khó khăn khi bước đầu làm việc trên server
- Có một thành viên không tập trung làm việc. Và quan trọng nhất là khối single_cycle, tổng số phần tử logic của khối single_cycle vượt quá tổng số phần tử logic của thiết bị Cyclone II. Điều này khiến chương trình không thể biên dịch trên Quartus.

6. Result

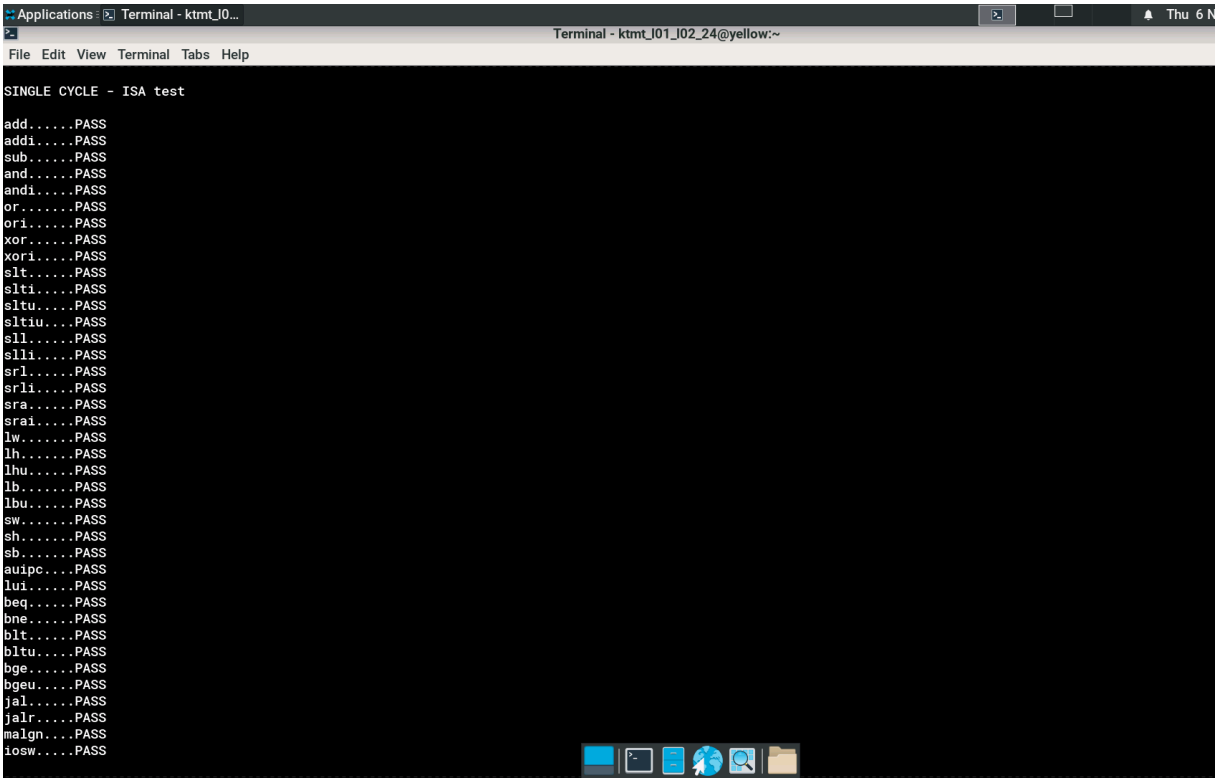
Lưu ý: Khi thiết kế theo yêu cầu bài tập với IMEM 8KiB và DMEM 2KiB, tổng số phần tử logic của khối chu kỳ đơn vượt quá tổng số phần tử logic của thiết bị Cyclone II. Điều này khiến chương trình không thể biên dịch trên Quartus. Do đó, nhóm chúng em đã giảm kích thước DMEM xuống còn 1KiB. Việc này sẽ không gây ra bất kỳ tác động đáng kể nào đến chức năng của toàn bộ khối.

6.1. Quartus (bổ sung module gán chân)

	Single cycle	**Cyclone II**	**Ratio**
Maximun frequency	29.5 MHz		

Logic Utilization/Total logic elements	23628	33216	71%
Total registers	9408	33216	28%

6.2. Server test



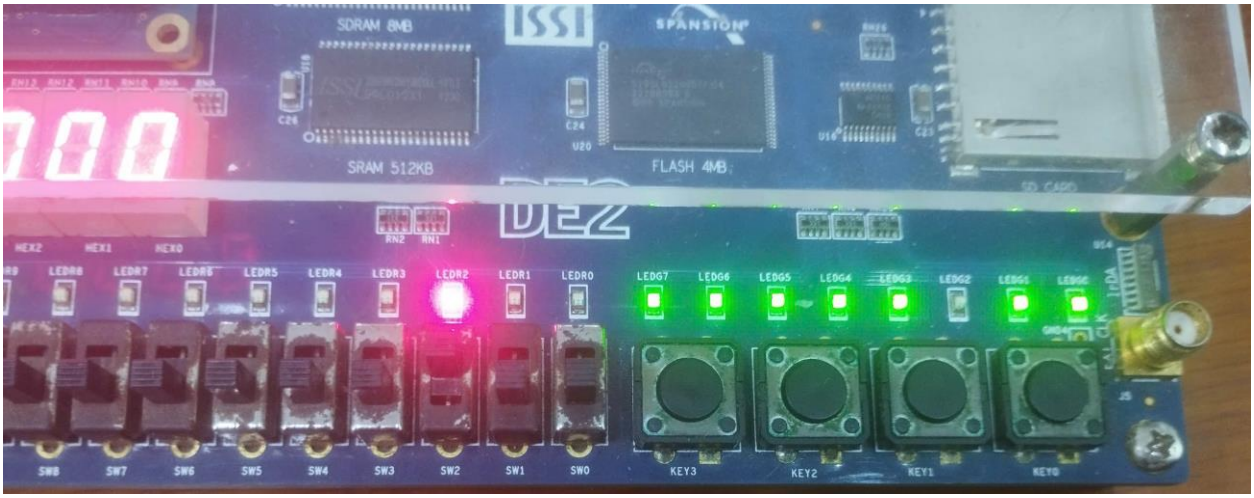
6.3. Applications

The team used different assembly programs to test the I/O peripherals of the single-cycle processor on the DE2 FPGA board. The results are as follows:

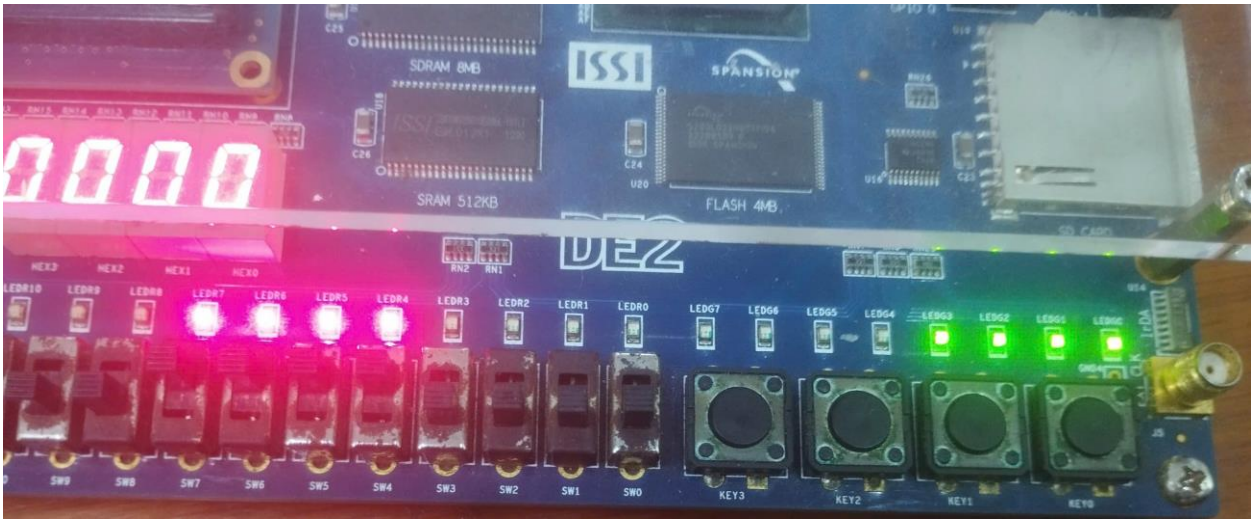
6.3.1. Test led red, led green and switches

Programme: Based on the status of switches, turn on the corresponding red LED and turn off the corresponding green LED.

- Case 1: Turn on SW2, result is LEDR2 is on and LEDG2 is off



- Case 2: Turn on SW7 to SW4, result is LEDR7 to LEDR4 are on and LEDG7 to LEDG4 are off.



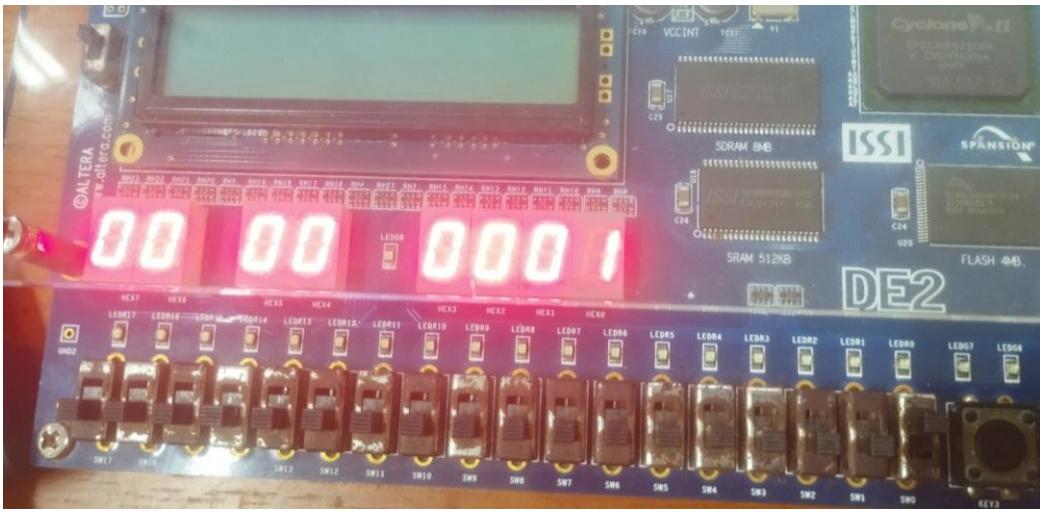
- Case 3: Press the button KEY0 to reset all the LEDs.

```
lui     x1, 65552 #sw
      lui       x2, 65536 #led red
      lui       x3, 65537 #led green
      lui    x4, 65538 #Hex 3-0
      lui    x5, 65539 #Hex 7-4
loop: lw     x6, 0(x1)
      sw        x6, 0(x2)
      xori      x7, x6, -1
      sw        x7, 0(x3)
      lui    x8, 0x30792
      addi   x8, x8, 0xFFFFF979
      sw     x8, 0(x5)
      lui    x8, 0x12102
      addi   x8, x8, 0x402
      sw     x8, 0(x4)
      jal       zero, loop #offset=-16
```

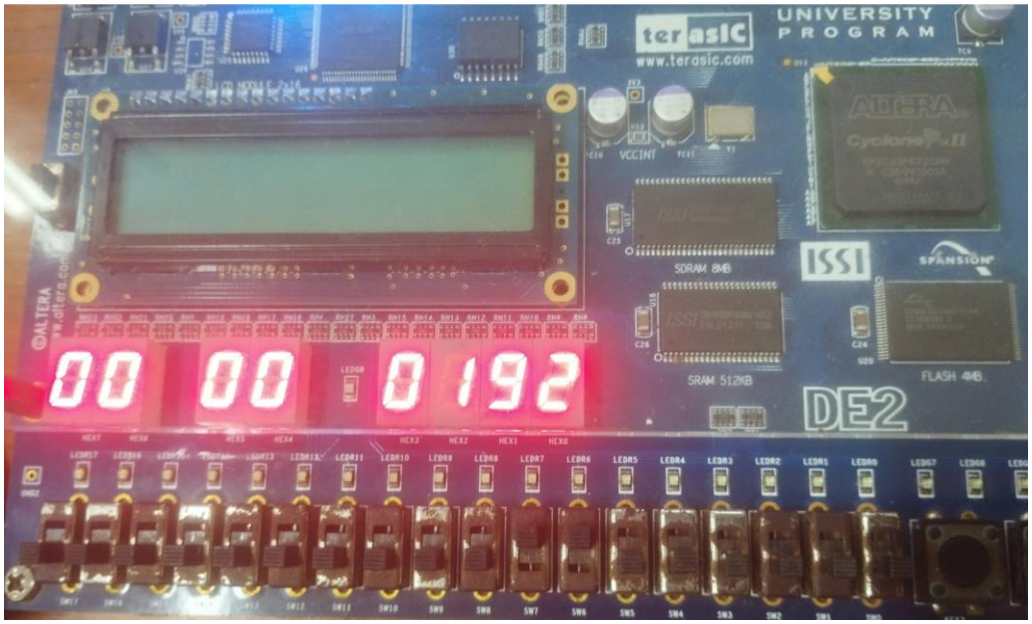
6.3.2. Test 7-segment LEDs

Programme: Convert a binary number to a decimal number and display on 7-segment LEDs. The values will be determined by switches.

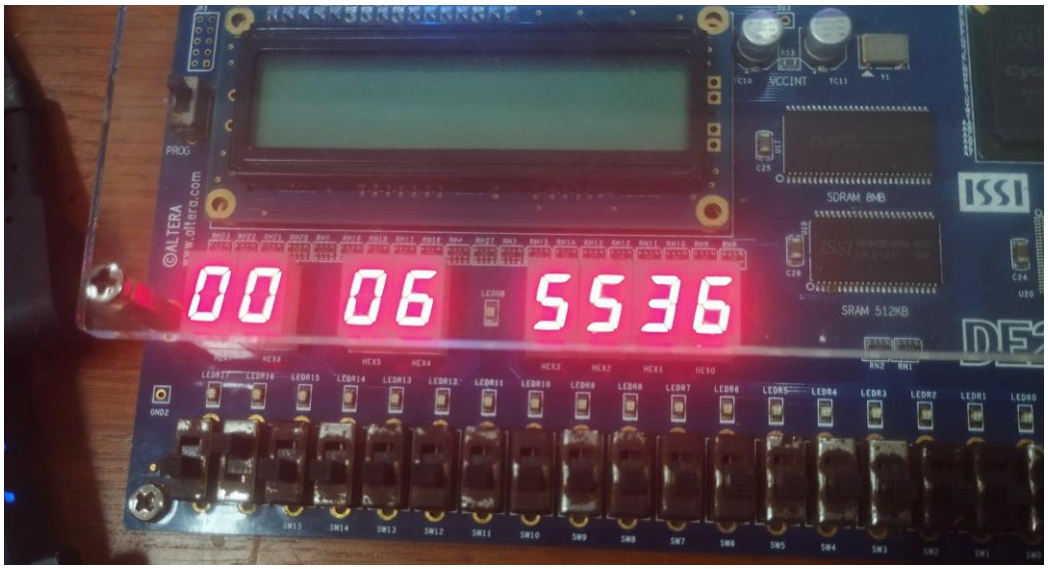
- Case 1: Turn on SW0, which corresponding to value 18’b00_0000_0000_0000_0001 = 1



- Case 2: Turn on SW7 and SW6, which corresponding to value 18’b00_0000_0000_1100_0000 = 192



- Case 3: Turn on SW16, which corresponding to value 18’b01_0000_0000_0000_0000 = 65536




```
lui  x10, 65552          # x10 = 0x10010000 (switches)
    lui  x11, 65538          # x11 = 0x10002000 (HEX3-0)
    lui  x12, 65539           # x12 = 0x10003000 (HEX7-4)
    #0
    addi x14,x0,0
    addi x13,x0, 0x40
    sb   x13,0(x14)
    #1
    addi x14,x0,1
    addi x13,x0, 0x79
    sb   x13,0(x14)
    #2
    addi x14,x0,2
    addi x13,x0, 0x24
    sb   x13,0(x14)
    #3
    addi x14,x0,3
    addi x13,x0, 0x30
    sb   x13,0(x14)
    #4
    addi x14,x0,4
    addi x13,x0, 0x19
    sb   x13,0(x14)
    #5
    addi x14,x0,5
    addi x13,x0, 0x12
    sb   x13,0(x14)
    #6
    addi x14,x0,6
    addi x13,x0, 0x02
    sb   x13,0(x14)
    #7
    addi x14,x0,7
    addi x13,x0, 0x78
    sb   x13,0(x14)
```

```
    #8
    addi x14,x0,8
    addi x13,x0, 0x00
    sb   x13,0(x14)
    #9
    addi x14,x0,9
    addi x13,x0, 0x10
    sb   x13,0(x14)

main_loop:
    # ============================================================
    # Đọc giá trị từ switches
    # ============================================================
    lw   x5, 0(x10)          # x5 = switches


    # ============================================================
    # Tách chữ số hàng nghìn (digit 5: chia cho 100000
    # ============================================================
    addi x23, x0, 0          # x23 = digit3 = 0
    addi x7,x0,0x01
    slli x7,x7,8
    addi x7,x0,0x86
    slli x7,x7,8
    addi x7,x7,0xA0

div_100000:
    blt  x5, x7, end_div_100000  # if x5 < 100000
    sub  x5, x5, x7              # x5 = x5 - 100000
    addi x25, x25, 1             # digit3++
    jal  x0, div_100000          # jump
end_div_100000:

    # ============================================================
    # Tách chữ số hàng nghìn (digit 4) chia cho 10000
    # ============================================================
    addi x23, x0, 0          # x23 = digit3 = 0
    addi x7,x0,0x00
    slli x7,x7,8
    addi x7,x0,0x27
    slli x7,x7,8
    addi x7,x7,0x10

div_10000:
    blt  x5, x7, end_div_10000  # if x5 < 10000
    sub  x5, x5, x7              # x5 = x5 - 10000
    addi x24, x24, 1             # digit3++
    jal  x0, div_10000          # jump
end_div_10000:

    # ============================================================
    # Tách chữ số hàng nghìn (digit 3): chia cho 1000
    # ============================================================
```

```
    addi x23, x0, 0          # x23 = digit3 = 0
    addi x7, x0, 1000        # x7 = 1000

div_1000:
    blt  x5, x7, end_div_1000    # if x5 < 1000, kết thúc
    sub  x5, x5, x7              # x5 = x5 - 1000
    addi x23, x23, 1             # digit3++
    jal  x0, div_1000            # jump
end_div_1000:
 addi x22, x0, 0          # x22 = digit2 = 0
    addi x7, x0, 100         # x7 = 100

div_100:
    blt  x5, x7, end_div_100
    sub  x5, x5, x7
    addi x22, x22, 1
    jal  x0, div_100
end_div_100:

    # ============================================================
# Tách chữ số hàng chục (digit 1): chia cho 10
    # ============================================================
    addi x21, x0, 0          # x21 = digit1 = 0
    addi x7, x0, 10          # x7 = 10

div_10:
    blt  x5, x7, end_div_10
    sub  x5, x5, x7
    addi x21, x21, 1
    jal  x0, div_10
end_div_10:

    # ============================================================
    # x5 = digit0 (hàng đơn vị)
    # ============================================================
    add  x20, x5, x0         # x20 = digit0

    #output

    sb x0,3(x12)
    sb x0,2(x12)

    lb x14,0(x25)
    sb x14,1(x12)
    lb x14,0(x24)
    sb x14,0(x12)

    lb x14,0(x23)
    sb x14,3(x11)
    lb x14,0(x22)
    sb x14,2(x11)
    lb x14,0(x21)
    sb x14,1(x11)
```

```
lb x14,0(x20)
sb x14,0(x11)
jal  x0, main_loop
```

6.3.3. Test LCD: Tìm điểm gần điểm tọa độ O nhất.

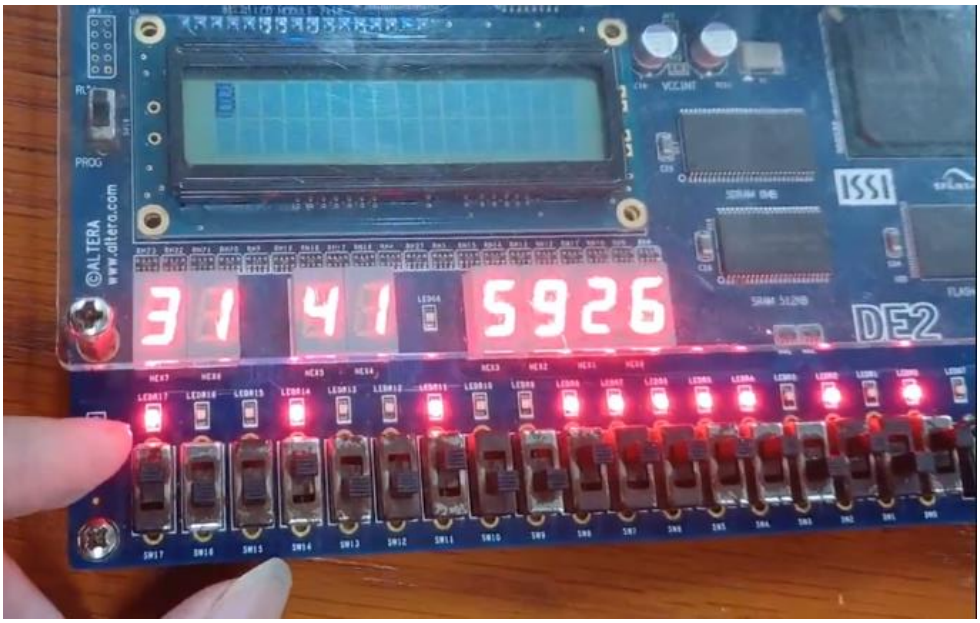
Trong ứng dụng này, chúng ta sử dụng SW0 đến SW3 để chọn tọa độ x và SW4 đến SW7 để chọn tọa độ y cho điểm A. Sử dụng SW8 đến SW11 để chọn tọa độ x và SW12 đến SW15 để chọn tọa độ y cho điểm B. Chúng ta sẽ xác định điểm A hay B gần O hơn bằng cách sử dụng màn hình LCD.

- Case 1:

  Đặt giá trị SW0 đến SW3 thành 4’b0101, SW4 đến SW7 thành 4’b1111. Tọa độ của điểm A là $(x_A, y_A) = (5; 15)$

  Đặt giá trị của SW8 to SW11 là 4’b1001, SW4 to SW7 là 4’b0100. Tọa độ của B là $(x_B, y_B) = (9; 4)$

  Theo mong đợi, point B gần O (0;0) hơn so với A.



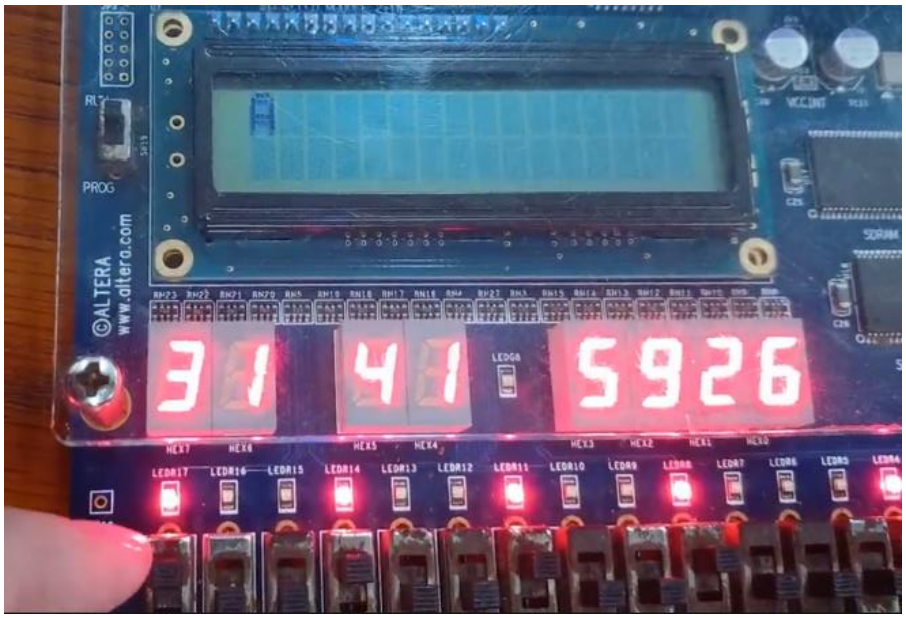
- Case 2:

  Đặt giá trị của SW0 to SW3 là 4’b0000, SW4 to SW7 là 4’b0001. Tọa độ

A là $(x_A, y_A) = (0; 1)$

Đặt giá trị của SW8 to SW11 là 4’b1001, SW4 to SW7 là 4’b0100. Tọa độ B là $(x_B, y_B) = (9; 4)$

Mong đợi, point A gần O hơn.




```
ON_LSD:
        lui   x10, 65540 #LCD
        addi  x9,x0,0x80
        sb    x9,3(x10)
IOs:
        lui x1, 65552 #sw
        lui x2, 65536 #led red
        lui x3, 65537 #led green
        lui   x4, 65538 #Hex 3-0
        lui   x5, 65539 #Hex 7-4
Start_LCD:
        addi  x9,x0,0x30 #start LCD 1
        sh    x9,0(x10)
        addi  x9,x0,0x430 #start LCD 1
        sh    x9,0(x10)
        addi  x9,x0,0x30
        sh    x9,0(x10)
        addi  x9,x0,0x30 #start LCD 2
        sh    x9,0(x10)
        addi  x9,x0,0x430 #start LCD 2
        sh    x9,0(x10)
        addi  x9,x0,0x30
        sh    x9,0(x10)
        addi  x9,x0,0x30 #start LCD 3
        sh    x9,0(x10)
```

```
      addi  x9,x0,0x430 #start LCD 3
      sh    x9,0(x10)
      addi  x9,x0,0x30
      sh    x9,0(x10)
Initial_LCD:
      addi  x9,x0,0x38  #write 0x38 command into LCD
      sh    x9,0(x10)
      addi  x9,x0,0x438  #write 0x38 command into LCD
      sh    x9,0(x10)
      addi  x9,x0,0x38  # 1-0 transmition of EN = io_lcd [10] perform access LCD
      sh    x9,0(x10)
      #next command for the LCD, do same as the above
      addi  x9,x0,0x01  #write 0x01 command into LCD
      sh    x9,0(x10)
      addi  x9,x0,0x401  #write 0x01 command into LCD
      sh    x9,0(x10)
      addi  x9,x0,0x01  # 1-0 transmition of EN = io_lcd [10] perform access LCD
      sh    x9,0(x10)
      #next command
      addi  x9,x0,0x0C  #write 0x0C command into LCD
      sh    x9,0(x10)
      addi  x9,x0,0x40C  #write 0x0C command into LCD
      sh    x9,0(x10)
      addi  x9,x0,0x0C  # 1-0 transmition of EN = io_lcd [10] perform access LCD
      sh    x9,0(x10)
      #next command
      addi  x9,x0,0x06  #write 0x06 command into LCD
      sh    x9,0(x10)
      addi  x9,x0,0x406  #write 0x06 command into LCD
      sh    x9,0(x10)
      addi  x9,x0,0x06  # 1-0 transmition of EN = io_lcd [10] perform access LCD
      sh    x9,0(x10)
LOOP:
     #FIND THE CLOSEST POINT TO O
     closest_point:
     lui x1, 0x10010 # đọc sw tương ứng tọa độ của A,B
     lw x11, 0(x1)
     andi x12, x11, 15 # lấy xA
     srli x16, x11, 4
     andi x13, x16, 15 # lấy yA
     srli x16, x11, 8
     andi x14, x16, 15 # lấy xB
     srli x16, x11, 12
     andi x15, x16, 15 # lấy yB
     #Tìm bình phương xA
     add x18, x0, x12
     jal x6, times
     add x12, x0, x20
     #Tìm bình phương yA
     add x18, x0, x13
     jal x6, times
     add x13, x0, x20
    #Tìm bình phương xB
```

```
    add x18, x0, x14
    jal x6, times
    add x14, x0, x20
    #Tìm bình phương yB
    add x18, x0, x15
    jal x6, times
    add x15, x0, x20
    #Tiến hành so sánh khoảng cách của A và B so với O
    add x18, x12, x13
    add x19, x14, x15
    bltu x18,x19,A
B:
    addi x20,x0,0x42
    jal x0,LCD_DISPLAY
A:
    addi x20,x0,0x41
LCD_DISPLAY:
      # point to 0x80 on the LCD (the start location of line 1 on the LCD display)
      addi  x9,x0,0x80  #write 0x80 command into LCD
      sh    x9,0(x10)
      addi  x9,x0,0x480  #write 0x80 command into LCD
      sh    x9,0(x10)
      addi  x9,x0,0x80  # 1-0 transmition of EN = io_lcd [10] perform access LCD
      sh    x9,0(x10)
 #-------------------------------------
      addi  x9,x20,0x200  #write 0x56 data into LCD#
      sh    x9,0(x10)
      addi  x9,x20,0x600  #write 0x56 data into LCD#
      sh    x9,0(x10)
      addi  x9,x20,0x200  # 1-0 transmition of EN = io_lcd [10] perform access LCD
      sh    x9,0(x10)
#=============================================
      lw    x6, 0(x1)
      sw  x6, 0(x2)
      xori         x7, x6, -1
      sw  x7, 0(x3)
      lui   x8, 0x30792
      addi  x8, x8, 0xFFFFF979
      sw    x8, 0(x5)
      lui   x8, 0x12102
      addi  x8, x8, 0x402
      sw    x8, 0(x4)
      jal zero,LOOP
#-------------------------------------
DELAY6ms:
       addi x12,x0,0x49
       slli x12,x12,12
       addi x12,x0,0x3E2
LOOP_DELAY6ms:
       addi x12,x12,-1
       addi x13,x0,2
       beq  x12,x13,END_DELAY6ms
       jal x0,LOOP_DELAY6ms
```

```
END_DELAY6ms:
       jalr x0,x15,0
#-------------------------------------
#-------------------------------------------
#Chương trình tìm bình phương của một số
#-------------------------------------------
    times:
    add x19, x0, x18
    add x20, x0, x0
    loop:
    beq x19, x0, end
    add x20, x20, x18
    addi x19, x19, -1
    j loop
    end: jr x6
```

6.3.4. Triển khai ứng dụng khai căn bậc hai một số 8bit (Ứng dụng chính của nhóm)

Video: https://drive.google.com/file/d/1DrtqN0gKlsMipK4VoqbLgVnn2mn7BGOu/view?usp=drivesdk

```
start:
      lui  x11,65552          # x10 = 0x10010000 (switches)
ON_LCD:
      lui  x10, 65540          # x11 = LCD address
      addi  x12,x0,0x80       # ON LCD
      sb    x12,3(x10)
     jal   x9,DELAY_>_6ms
     jal   x9,DELAY_>_6ms
     jal   x9,DELAY_>_6ms
     jal   x9,DELAY_>_6ms
     jal   x9,DELAY_>_6ms
     jal   x9,DELAY_>_6ms
Start_LCD:
      #start LCD 1
      addi  x12,x0,0x30
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x430
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x30
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      #start LCD 2
      addi  x12,x0,0x30
      sh    x12,0(x10)
```

```
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x430
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x30
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
     #start LCD 3
      addi  x12,x0,0x30
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x430
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x30
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
Initial_LCD:
      addi  x12,x0,0x38  #write 0x38 command into LCD
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x438
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x38  # 1-0 transmition of EN = io_lcd [10] perform access LCD
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      #next command for the LCD, do same as the above
      addi  x12,x0,0x01  #write 0x01 command into LCD
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x401
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x01  # 1-0 transmition of EN = io_lcd [10] perform access LCD
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      #next command
      addi  x12,x0,0x0C  #write 0x0C command into LCD
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x40C
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x0C  # 1-0 transmition of EN = io_lcd [10] perform access LCD
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      #next command
      addi  x12,x0,0x06  #write 0x06 command into LCD
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x406
      sh    x12,0(x10)
```

```
      jal   x9,DELAY_>_6ms
      addi  x12,x0,0x06  # 1-0 transmition of EN = io_lcd [10] perform access LCD
      sh    x12,0(x10)
      jal   x9,DELAY_>_6ms
    #--------------
    #save the value decimal with binary index -8 to -1
    #391
    addi x14,x0,0
    addi x13,x0,391
    sh   x13,0(x14)
    #781
    addi x14,x0,2
    addi x13,x0,781
    sh   x13,0(x14)
    #1563
    addi x14,x0,4
    addi x13,x0,1563
    sh   x13,0(x14)
    #3125
    addi x14,x0,6
    addi x13,x0,0x0C
    slli x13,x13,8
    addi x13,x13,0x35
    sh   x13,0(x14)
    #6250
    addi x14,x0,8
    addi x13,x0,0x18
    slli x13,x13,8
    addi x13,x13,0x6A
    sh   x13,0(x14)
    #12500
    addi x14,x0,10
    addi x13,x0,0x30
    slli x13,x13,8
    addi x13,x13,0xD4
    sh   x13,0(x14)
    #25000
    addi x14,x0,12
    addi x13,x0,0x61
    slli x13,x13,8
    addi x13,x13,0xA8
    sh   x13,0(x14)
    #50000
    addi x14,x0,14
    addi x13,x0,0xC3
    slli x13,x13,8
    addi x13,x13,0x50
    sh   x13,0(x14)
MAIN:
      lbu'    x16,0(x11)  #load the data that is stored on the sw reg
      slli  x16,x16,16
      addi x15,x0,-1
LOOP:
```

```
      addi x15,x15,1
      addi x12,x15,0
      jal  x9,POWERS_OF_2
      bltu  x12,x16,LOOP
      beq  x12,x16,NEXT_HANLE
      addi x15,x15,-1
NEXT_HANLE: # decompose to the number characters of the result
      addi x12,x0,0xF
      slli x12,x12,8
      and x19,x15,x12 #filter the interger part and stored in x19 #
      srli x19,x19,8
      addi x12,x0,0xFF
      and x18,x15,x12 #filter the decimal part and stored in x18 #
##########################################################################################
#
#### add the decimal value that is corresponding with the binary indexs that is the "1"
characters  ###
##########################################################################################
#
      addi x13,x0,8
      addi x15,x0,0
LOOP_DECIMAL_POINT_HANDLE:
      addi x13,x13,-1
      blt x13,x0,END_DECIMAL_POINT_HANDLE
      addi x14,x0,1
      sll x14,x14,x13
      and x12,x18,x14
      beq  x12,x0, LOOP_DECIMAL_POINT_HANDLE
      add x17,x13,x13
      lhu  x12,0(x17)
      add x15,x15,x12
      jal x0,LOOP_DECIMAL_POINT_HANDLE
END_DECIMAL_POINT_HANDLE:
      addi x18,x15,0     #store the add result into the x18
BCD_LEFT_SIDE_OF_THE_POINT:
      addi x12,x0,9
      sltu  x26,x12,x19
      beq  x26,x0,LESS_THAN_10
      addi x25,x19,-10
      jal x0,BCD_RIGHT_SIDE_OF_THE_POINT
LESS_THAN_10:
      addi x25,x19,0
BCD_RIGHT_SIDE_OF_THE_POINT:
      #div 10000
      addi x13, x0,0x27
      slli x13,x13,8
      addi x13,x13,0x10 #10000
      addi x14,x18,0
      jal x9,div_function
      addi x24,x12,0
      #div 1000
      addi x13, x0,1000
      addi x14,x14,0
```

```
        jal x9,div_function
        addi x23,x12,0
        #div 100
        addi x13, x0,100
        addi x14,x14,0
        jal x9,div_function
        addi x22,x12,0
        #div 10
        addi x13, x0,10
        addi x14,x14,0
        jal x9,div_function
        addi x21,x12,0
        addi x20,x14,0 #mod
LCD_display:
        # point to 0x80 on the LCD (the start location of line 1 on the LCD display)
        addi  x12,x0,0x80  #write 0x80 command into LCD
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x0,0x480
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x0,0x80  # 1-0 transmition of EN = io_lcd [10] perform access LCD
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        #write data "R26"
        addi  x12,x26,0x230
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x26,0x630
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x26,0x230  # 1-0 transmition of EN = io_lcd [10] perform access LCD
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        #write data "R25"
        addi  x12,x25,0x230
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x25,0x630
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x25,0x230  # 1-0 transmition of EN = io_lcd [10] perform access LCD
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        #write data "."
        addi  x12,x0,0x22E
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x0,0x62E
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x0,0x22E  # 1-0 transmition of EN = io_lcd [10] perform access LCD
        sh    x12,0(x10)
```

```
        jal   x9,DELAY_>_6ms
        #write data "R24"
        addi  x12,x24,0x230
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x24,0x630
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x24,0x230  # 1-0 transmition of EN = io_lcd [10] perform access LCD
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        #write data "R23"
        addi  x12,x23,0x230
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x23,0x630
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x23,0x230  # 1-0 transmition of EN = io_lcd [10] perform access LCD
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        #write data "R22"
        addi  x12,x22,0x230
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x22,0x630
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x22,0x230  # 1-0 transmition of EN = io_lcd [10] perform access LCD
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        #write data "R21"
        addi  x12,x21,0x230
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x21,0x630
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x21,0x230  # 1-0 transmition of EN = io_lcd [10] perform access LCD
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        #write data "R20"
        addi  x12,x20,0x230
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x20,0x630
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
        addi  x12,x20,0x230  # 1-0 transmition of EN = io_lcd [10] perform access LCD
        sh    x12,0(x10)
        jal   x9,DELAY_>_6ms
END: jal x0,MAIN
```

```
#-------------------------------------
DELAY_>_6ms:
        addi x12,x0,0x49
        slli x12,x12,12
        addi x12,x12,0x3E0
LOOP_DELAY6ms:
        addi x12,x12,-1
        bge  x12,x0,LOOP_DELAY6ms
END_DELAY6ms:
        jalr x0,x9,0
#-------------------------------------
POWERS_OF_2:
        addi x14,x0,0
        addi x13,x12,0
        beq  x13,x0,END_POWER
LOOP_POWER:
        add x14,x14,x12
        addi x13,x13,-1
        beq x13,x0,END_POWER
        jal x0,LOOP_POWER
END_POWER:
        addi x12,x14,0
        jalr x0,x9,0
#-------------------------------------
div_function: #x12 is the result and x14 = x14 mod x13
    addi x12, x0, 0
LOOP_DIV:
    bltu  x14, x13, end_div_function
    sub  x14, x14, x13
    addi x12, x12, 1
    jal  x0, LOOP_DIV
end_div_function:
    jalr x0,x9,0
```

Ý tưởng giải thuật: Giả sử Ta tìm $\sqrt{a}$ = b suy ra a = b^2. Do a là một số 8 bit nên a chỉ thuộc từ 0 đến 255, nên giá trị b có tối đa 4 bit phần nguyên. Giả sử ta chọn độ phân giải của b là 8 bit thập phân. Ta có:

(b1b2b3b4.c1c2c3c4c5c6c7c8)^2 = a (trong đó các bit b là phần nguyên, c là phần thập phân). Ta dịch dấu chấm thập phân, ta có:

(b1b2b3b4c1c2c3c4c5c6c7c8 * 2^-8)^2 = a

Suy ra: (b1b2b3b4c1c2c3c4c5c6c7c8)^2 = a* 2^16

Vậy ta chỉ cần dịch phải đầu vào a đi 16 bit và xây dựng các hàm mũ và vòng lập

để tìm b1b2b3b4c1c2c3c4c5c6c7c8. Sau đó ta xử lý in ra LCD phần thập phân, phần nguyên tương ứng.

Tần số CPU chọn ở f = 50KHz. Các chương trình delay luôn đảm bảo >6ms để đảm bảo LCD hoạt động trơn tru (6ms luôn đảm bảo thỏa các hoạt động timing của LCD trong datasheet).

References

Lastname, C. (2008). Title of the source without caps except Proper Nouns or: First word after colon. *The Journal or Publication Italicized and Capped*, Vol#(Issue#), Page numbers.

Lastname, O. (2010). Online journal using DOI (digital object identifier). *Main Online Journal Name*, Vol#(Issue#), 159-192. https://doi.org/10.1000/182